Second half of FY2024 SOKEN-Dai

Simulation Astronomy
(Computational Astrophysics)

Junichi Baba

(D. of Science / Kagoshima Univ. AGARC)

Contact: jun.baba@nao.ac.jp

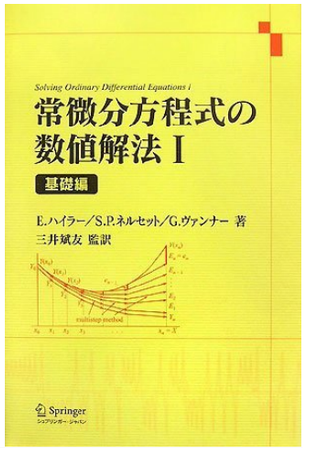
References

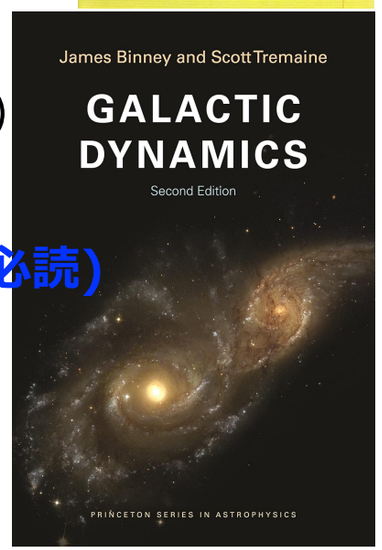
An Introduction to Computational Astrophysics:

- **シリーズ現代の天文学14巻　シミュレーション天文学、富阪 他、日本評論社**

Basics of ODEs, Numerical Methods:

- Advanced Engineering Mathematics, Erwin Kreyszig (1999)
  (邦訳：技術者のための高等数学１ 常微分方程式、クライツィグ、培風館)
- **Solving Ordinary Differential Equations I, E. Hairer et al.**
  (邦訳：常微分方程式の数値解析 I 基礎編、ハイラー 他、シュプリンガー・ジャパン)





Stellar Dynamics (galaxies, star clusters):

- **Galactic Dynamics 2nd, Binney & Tremaine, Princeton, 2008 <— Bible! (必読)**
- Dynamical Evolution of Globular Clusters, Spitzer, Princeton, 1987
- The Gravitational Million-Body, Heggie & Hut, Cambridge, 2003

Classical Mechanics:

- **Mechanics, Landau & Lifshitz (邦訳：力学、ランダウ・リフシッツ、東京図書)**
- Classical Mechanics, Goldstein (邦訳：古典力学 上・下、ゴールドスタイン、吉岡書店)

Lecture #1 (2024/10/25)

Numerical Methods for Ordinary Differential Equations

（常微分方程式の数値解法）

1. Finite Difference Method (有限差分法)
2. Runge-Kutta Method (ルンゲ・クッタ法)
3. Multistep Methods (多段階法)
4. Predictor-Corrector Methods (予測子・修正子法)

What is *Computational* Astrophysics?

- **Astronomical Phenomena**

  Observable but most cannot be reproduced in a laboratory.

Computer Simulation (計算機実験、数値実験)

Recreating and observing natural phenomena within a computer to understand astronomical phenomena. (自然現象を計算機の中に再現し、観測することで、天文現象を理解する)

Procedure :

1. **Modelling** the system and **solving** fundamental physical equations
   e.g., equations of motion, fluid dynamics, thermodynamics
2. **Convert** the obtained **physical quantities** (e.g., position, velocity, density, temperature, chemical composition) into **observables** (e.g., parallax, proper motion, radial velocity, radiation intensity, spectral lines)
   = *pseudo-observation* (擬似観測) / observational visualization (観測的可視化)
3. **Compare** the pseudo-observation results from the simulation **with actual observations**.

Ex) Milky Way galaxy simulations

Baba 2015; Baba et al. 2009, 2010, 2017, 2022

Galaxy Evolution

= Nonlinear Phenomenon in a Multi-Component System

✓Hundreds of billions of stars + diverse interstellar gas (molecular/atomic/ionized) + dark matter

✓Coupling of gravitational dynamics and matter changes

Diverse Physical Processes :

✓**Collisionless dynamics** (stars, DM) + **Hydrodynamics** (gas)

✓**Self-gravitational interactions** among stars, gas, and dark matter

✓**Thermal evolution of ISM** due to radiative cooling/heating ($10<T<10^8$ K)

✓**Star formation** from molecular clouds (~10 K) (gas → stars)

✓**Feedback** from HII regions and supernova explosions (stars → gas)

Wide Time and Spatial Scales (>4 orders) :

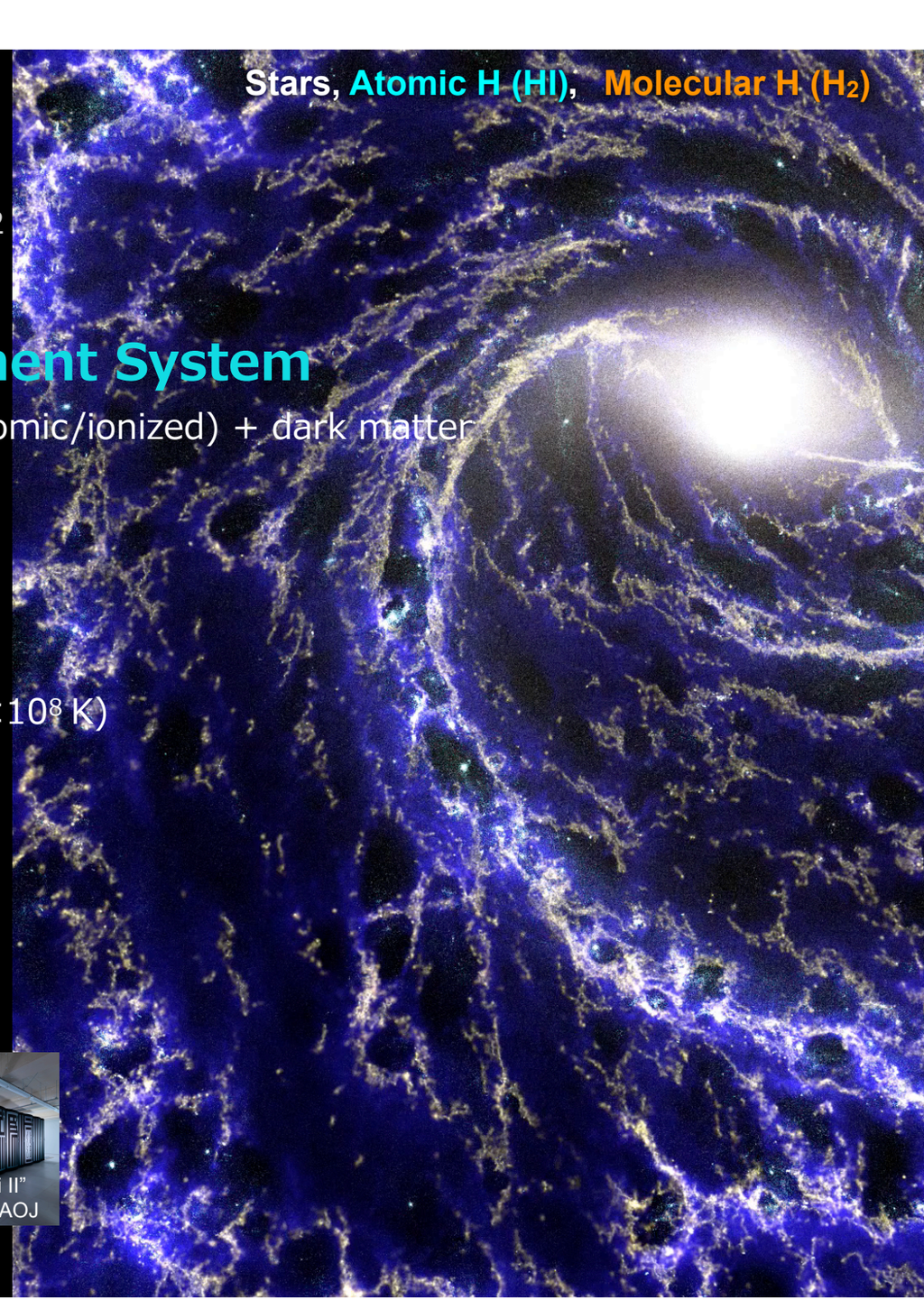
✓Star clusters / gas clouds (~pc) ↔ galactic structures (~10 kpc)

✓Clouds (~10^5 yr) ⟷ Galaxy (~10^9 yr)

Importance of Simulations :

✓Fast computers (e.g., ATERUI II)

✓Efficient algorithms (e.g., ASURA code)



What is *Computational* Astrophysics?

- **Astronomical Phenomena**

  Observable but most cannot be reproduced in a laboratory.

Computer Simulation (計算機実験、数値実験)

Recreating and observing natural phenomena within a computer to understand astronomical phenomena. (自然現象を計算機の中に再現し、観測することで、天文現象を理解する)

Procedure :

1. **Modelling** a target system and **solving** fundamental physical equations

   e.g., equations of motion, fluid dynamics, thermodynamics

2. **Convert** the obtained **physical quantities** (e.g., position, velocity, density, temperature, chemical composition) into **observables** (e.g., parallax, proper motion, radial velocity, radiation intensity, spectral lines)

   = *pseudo-observation* (擬似観測) / observational visualization (観測的可視化)

3. **Compare** the pseudo-observation results from the simulation **with actual observations**.

Modelling

- 離散的な変数の集合 = **粒子モデル (Particle Models)**

  e.g. Stellar systems (galaxies, dark matter, star clusters), Planetary systems, Plasma

- 連続的な場 = **流体モデル/連続体モデル (Fluid / Continuum Models)**

  e.g. Interstellar gas, Protoplanetary disks, Accretion disks

  Note: Even for fluids, it is possible to model them as collections of particles and define physical quantities by smoothing over many particles. This approach is known as **Smoothed Particle Hydrodynamics (SPH)**.
  (Section 4~7 by Iwasaki-san)

Classification of Differential Equations

- **Ordinary Differential Equation, ODE (常微分方程式)**
  - Particle model (粒子モデル)
  - ODE involves derivatives with respect to a single independent variable, often time (t). $\frac{dy(t)}{dt} = f(t, y(t))$
  - Equations of motion for stars or dark matter, chemical reactions
- **Partial Differential Equation, PDE (偏微分方程式)**
  - Fluid model (流体モデル)
  - PDE involves partial derivatives with respect to two or more independent variables (e.g., time t, space x).

    $$\frac{\partial u(x,t)}{\partial t} = \alpha \frac{\partial^2 u(x,t)}{\partial x^2}$$
  - hydrodynamics eq., Wave eq., Heat conduction eq., Maxwell's eq.

Classification of ODE Problems

- **Initial Value Problem (初期値問題)**
  - A problem where the initial state of the solution to the ODE is given.

$$\frac{dy(t)}{dt} = f(t, y(t)), \quad y(t_0) = y_0$$

  - e.g. free fall motion, ballistic motion, and chemical reactions
- **Boundary Value Problem (境界値問題)**
  - A problem where the solution to the ODE is given boundary conditions.

$$\frac{d^2y(x)}{dx^2} = f(x, y), \quad y(a) = A, \quad y(b) = B$$

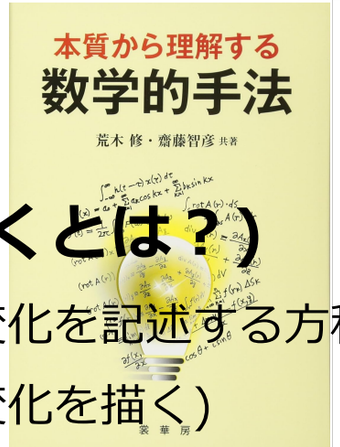
  - e.g. the shape of a chain hanging from the ceiling (天井につるした鎖の形状)
- **Eigen Value Problem (固有値問題)**
  - A problem where the solution to the ODE depends on a specific parameter λ (eigenvalue), under certain conditions.

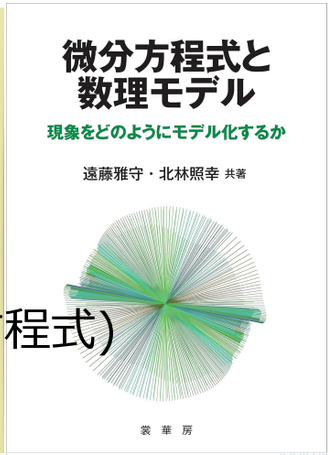
$$\frac{d^2y(x)}{dx^2} + \lambda y(x) = 0, \quad y(a) = 0, \quad y(b) = 0$$

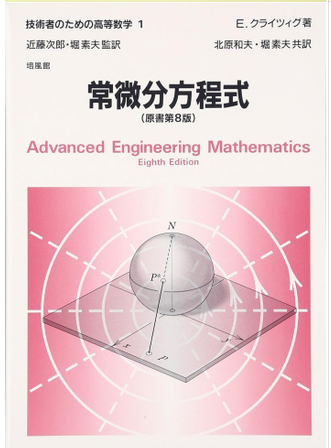
  - e.g. Vibration modes of a musical instrument, the steady-state Schrödinger equation.

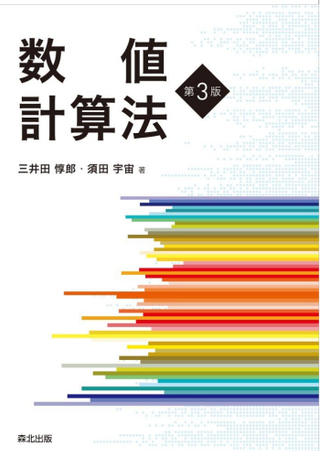
Solving ODEs

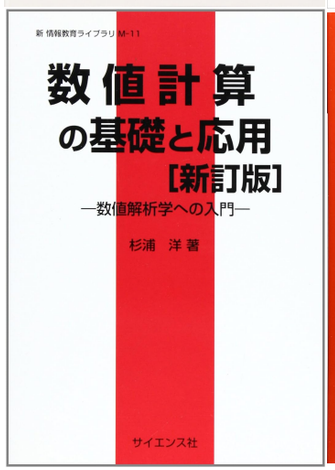
- **What does solving an ODE mean? (微分方程式を解くとは？)**
  - ODEs = how a system's state changes over time. (状態の時間変化を記述する方程式)
  - Solving an ODE = trajectory in space (空間内に解の時間的な軌道変化を描く)
  - Goal: finding the unknown function that satisfies the ODE
    (常微分方程式を満たす未知の関数を求めること)
- **Solving Analytically (解析的に解く)**
  - e.g. Variation of Parameters (定数変化法)
  - Linear ODEs: often possible (解析解が求まる可能性がある)
  - Non-Linear ODEs: almost impossible (解析解を求めるのは困難)
- **Solving Numerically (数値的に解く)**
  - Discretizing ODEs (微分方程式を離散化)
  - General: Euler, Runge-Kutta, Multi-step …etc.
  - *N*-body: **Leap-frog, Hermite**











Basic Concept: Finite Difference Method

Initial Value Problem:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

Discretization (離散化): 連続的数値を離散的数値で近似

- Approximating continuous values with discrete values
- Divide the closed interval $[x_0, x_N]$ into N parts, including the initial value.
  (初期値を含む閉区間$[x_0, x_N]$をN分割する)
- Compute the approximate solution y_n at each time $t_n = t_0 + \sum_{n=0}^{n=N-1} h_n$ sequentially.
  (各時刻 $t_n = t_0 + \sum_{n=0}^{n=N-1} h_n$ における解 $y(t_n)$ の近似解 y_n を逐次的に求める)

Finite Difference Method, FDM (有限差分法):

- derivatives dy/dt are approximated using differences between function values at discrete points

Other methods :

e.g. Finite Element Method (有限要素法), Finite Volume Method (有限体積法), Spectral Method (スペクトル法)

Euler Methods: The Simplest FDM

Using functions values y_n, y_{n+1} at a small interval $t = t_n$ and $t_n + h$, the following methods are defined:

i) $\frac{dy}{dt} = \frac{y_{n+1} - y_n}{h}$: **Forward Difference (前進差分)**

$y_{n+1} = y_n + h \cdot f(t_n, y_n)$: **Forward Euler (explicit)**

using only the current step (n).

ii) $\frac{dy}{dt} = \frac{y_n - y_{n-1}}{h}$: **Backward Difference (後退差分)**

$y_n = y_{n-1} + h \cdot f(t_n, y_n)$

$\rightarrow y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$ **: Backward Euler (implicit)**

using information from both the current step (n) and the **one-step future** ($n+1$)

iii) $\frac{dy}{dt} = \frac{y_{n+1} - y_{n-1}}{2h}$: **Central Difference (中心差分) ※ unstable**

$y_{n+1} = y_{n-1} + 2h \cdot f(t_n, y_n)$: **Central Difference Method (Explicit; two-step)**

using information from both the current step (n) and the **one-step past** ($n-1$)

Numerical Methods for Solving ODEs

	一段階法 **Single-step method**	**多段階法** **Multi-step method**
陽的公式 **Explicit formula**	Forward Euler, RK2, **RK4**, **Leap-frog**, **Hermite**	Midpoint, **Adams-Bashforth**
陰的公式 **Implicit formula**	Backward Euler, Implicit RK2, RK4	**BDF,** **Adams-Moulton**

Note: Forward/Backward Euler methods form the basis of the finite difference method, but it has low accuracy and is “not” practical for most applications.

Error Analysis of Euler Method: Taylor Expansion

The true solution, $y(t_{n+1})$, Taylor expansions around $t = t_n$:

$$y(t_{n+1}) = y(t_n + h) = y(t_n) + h \cdot y'(t_n) + \frac{h^2}{2} y''(t_n) + O(h^3) = \underline{y(t_n) + h \cdot f(t_n, y_n)} + O(h^2)$$

$$= y_{n+1} \text{ (Forward Euler)}$$

$\rightarrow$ Diff. btw the true and approximate solutions : $y(t_{n+1}) - y_{n+1} = O(h^2)$

2nd order accuracy
at each step
(i.e. Local Error)

Total number of steps: $N_{\text{step}} \propto \frac{1}{h}$

Cumulative Error: $O(h^2) \times N_{\text{step}} \propto O(h^2)\frac{1}{h} = O(h)$

Fwd Euler method is **1st-order** accurate.

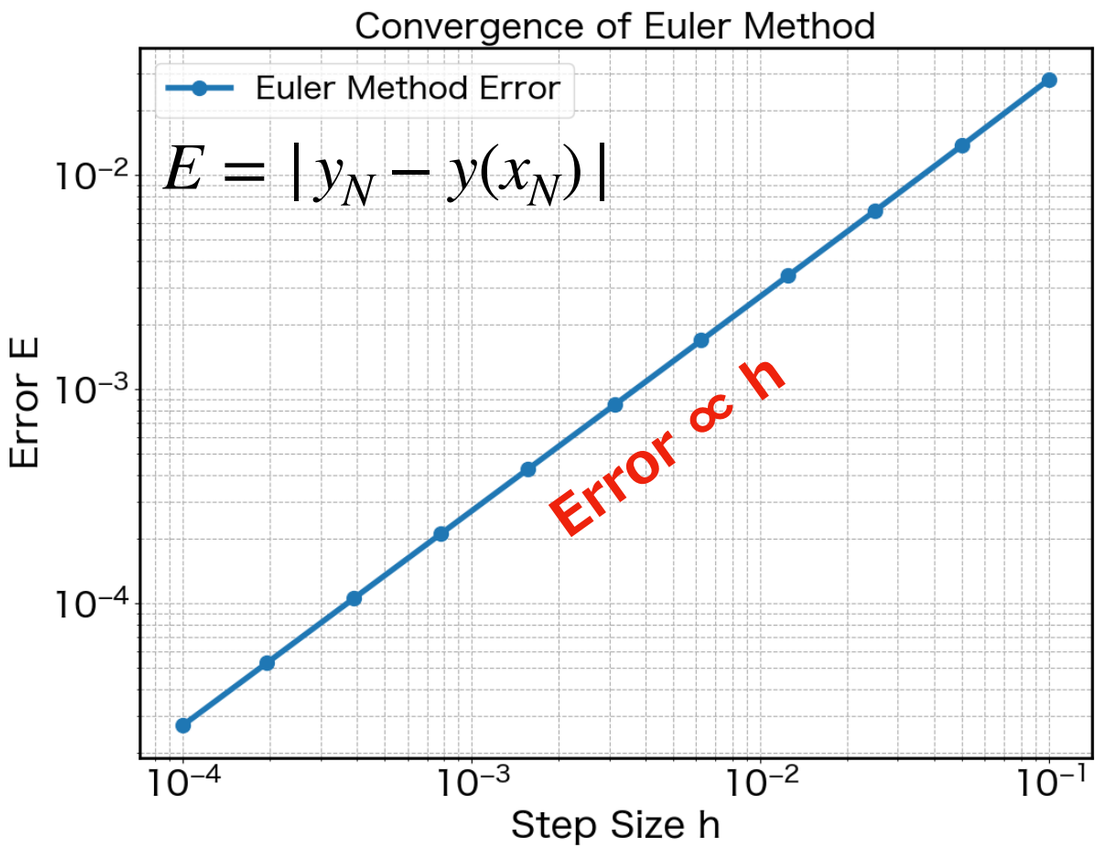
Ex.01) Euler Method : Linear ODE

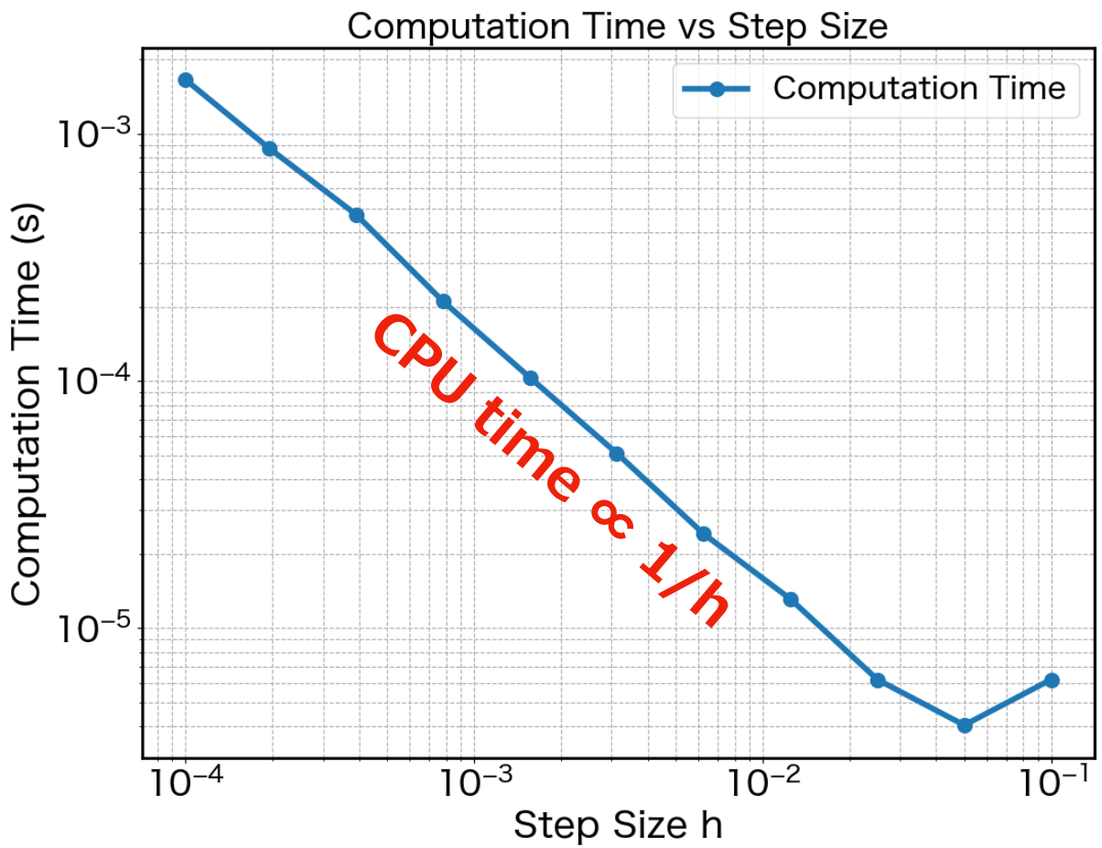
$$\frac{dy}{dt} = -2y, \quad y(0) = 1$$

analytic solution: $y(t) = e^{-2t}$

The numerical solution converges to the true solution with **an error** $\propto h$**.**

※ 収束性に関する数学的な証明は、三井やハイラーの教科書参照





Order of Integration Formulae

***p*-th order formula (*p*次公式)**

Local Truncation Error (局所誤差):

truncation error per an integration step: $O(h^{p+1})$

$$y(x_{i+1}) - y_{i+1} = O(h^{p+1})$$

$y(x_{i+1})$: exact solution, y_{i+1}: numerical solution

Global Truncation Error (大局誤差/累積誤差) :

cumulative truncation error for a certain integration interval

$$O(h^{p+1})\frac{1}{h} = O(h^p)$$

Towards Higher-Order Integrators

Euler's Method: Low accuracy, "not" practical for applications.

Need for Higher Accuracy Methods

- **Approach (1): Runge-Kutta Methods**
  - 2nd-order Runge-Kutta Methods: Heun method, Modified Euler method
  - 4th-order Runge-Kutta Method: Classical Runge-Kutta method
- **Approach (2): Multistep Methods and Predictor-Corrector Methods**
  - Adams-Bashforth Method (Explicit), Adams-Moulton Method (Implicit)
  - Predictor-Corrector Methods

Approach (1) : Runge-Kutta Methods

The method aims to achieve a higher-order Taylor expansion by calculating the function values of the derivative $f(t, y)$ at several intermediate points between $t = t_n$ and $t_n + h$, and then using an appropriate average of these values.

Ex. Using the value at one intermediate point（→ 2nd order formula）

$$y_{n+1} = y_n + \mu_1 hf(t_n, y_n) + \mu_2 hf(t_n + \alpha h, y_n + \beta k)$$

Chage $hf(t_n, y_n)$ based on the slope at the starting point (t_n, y_n), multiplied by the weight μ_1.
始点(t_n, y_n)での傾きによる変化量 $hf(t_n, y_n)$ を重み μ_1 を掛けた量

Chage $hf(t_n + \alpha h, y_n + \beta k)$ based on the slope at the intermediate point $(t_n + \alpha h, y_n + \beta k)$, multiplied by the weight μ_2.
中間点$(t_n + \alpha h, y_n + \beta k)$での傾きによる変化量を重み μ_2 を掛けた量

Taylor expansion : $f(t_n + \alpha h, y_n + \beta k) = f_n + \alpha h \left(\frac{\partial f}{\partial t}\right)_{t=t_n} + \beta k \left(\frac{\partial f}{\partial y}\right)_{t=t_n} + O(h^2) + O(k^2)$

$$= y_n + \mu_1 hf(t_n, y_n) + \mu_2 h \left[f_n + \alpha h \left(\frac{\partial f}{\partial t}\right)_{t=t_0} + \beta k \left(\frac{\partial f}{\partial y}\right)_{t=t_n} \right] + O(3)$$

$$\rightarrow y_{n+1} = y_n + (\mu_1 + \mu_2) hf(t_n, y_n) + \alpha\mu_2 h^2 \left(\frac{\partial f}{\partial t}\right)_{t=t_n} + \beta\mu_2 hk \left(\frac{\partial f}{\partial y}\right)_{t=t_n}$$ 2nd order

Determine the coefficients (α, β) and weights (μ_1, μ_2) so that the approximate solution matches the exact solution up to the 2nd-order term in the Taylor expansion (真の解とテイラー展開で2次の項まで一致するように係数と重みを決める)

Exact Solution

$$y(t_n + h) = y(t_n) + h\left(\frac{dy}{dt}\right)_{t=t_n} + \frac{1}{2}h^2\left(\frac{d^2y}{dt^2}\right)_{t=t_n} + O(h^3)$$

$$= y(t_n) + hf(t_n, y_n) + \frac{1}{2}h^2\left(\frac{df}{dt}\right)_{t=t_n} + O(h^3)$$

$$= y(t_n) + hf(t_n, y_n) + \frac{1}{2}h^2\left[\left(\frac{\partial f}{\partial t}\right)_{t=t_n} + \left(\frac{\partial f}{\partial y}\right)_{t=t_n}\left(\frac{dy}{dt}\right)_{t=t_n}\right] + O(h^3)$$

$$= y(t_n) + hf(t_n, y_n) + \frac{1}{2}h^2\left(\frac{\partial f}{\partial t}\right)_{t=t_n} + \frac{1}{2}h^2 f(t_n, y_n)\left(\frac{\partial f}{\partial y}\right)_{t=t_n} + O(h^3)$$

Approx. Solution

$$y_{n+1} = y_n + (\mu_1 + \mu_2)hf(t_n, y_n) + \alpha\mu_2 h^2\left(\frac{\partial f}{\partial t}\right)_{t=t_n} + \beta\mu_2 hk\left(\frac{\partial f}{\partial y}\right)_{t=t_n}$$

If $\mu_1 + \mu_2 = 1$, $\alpha\mu_2 = \beta\mu_2 = 1/2$, and $k = h \cdot f_n$, the exact solution and the approximate solution satisfy $y(t_n + h) - y_{n+1} = O(h^3)$. Local and global truncation errors are $O(h^3)$ and $O(h^2)$, respectively.

2nd order Runge-Kutta (RK2) methods

The given approximate solution formula: $y_{n+1} = y_n + \mu_1 hf(t_n, y_n) + \mu_2 hf(t_n + \alpha h, y_n + \beta k)$ with conditions for second-order accuracy: $\mu_1 + \mu_2 = 1$、$\alpha\mu_2 = \beta\mu_2 = 1/2$、$k = hf_n$ involves four variables (α, β, μ_1, μ_2) and three equations, meaning there is one degree of freedom and **no unique solution**.

1) $\alpha = \beta = 1$、$\mu_1 = \mu_2 = 1/2$:

$$y_{n+1} = y_n + \frac{1}{2}h\left[f(t_n, y_n) + f(t_n + h, y_n + h \cdot f_n)\right]$$

Heun method (ホイン法)
or Improved Euler method
(改良オイラー法)

2) $\alpha = \beta = 1/2$、$\mu_1 = 0$、$\mu_2 = 1$:

$$y_{n+1} = y_n + h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f_n\right)$$

Modofiled Euler method
(修正オイラー法)

RK2 methods: Algorithm

Heun Method (Improved Euler Method)

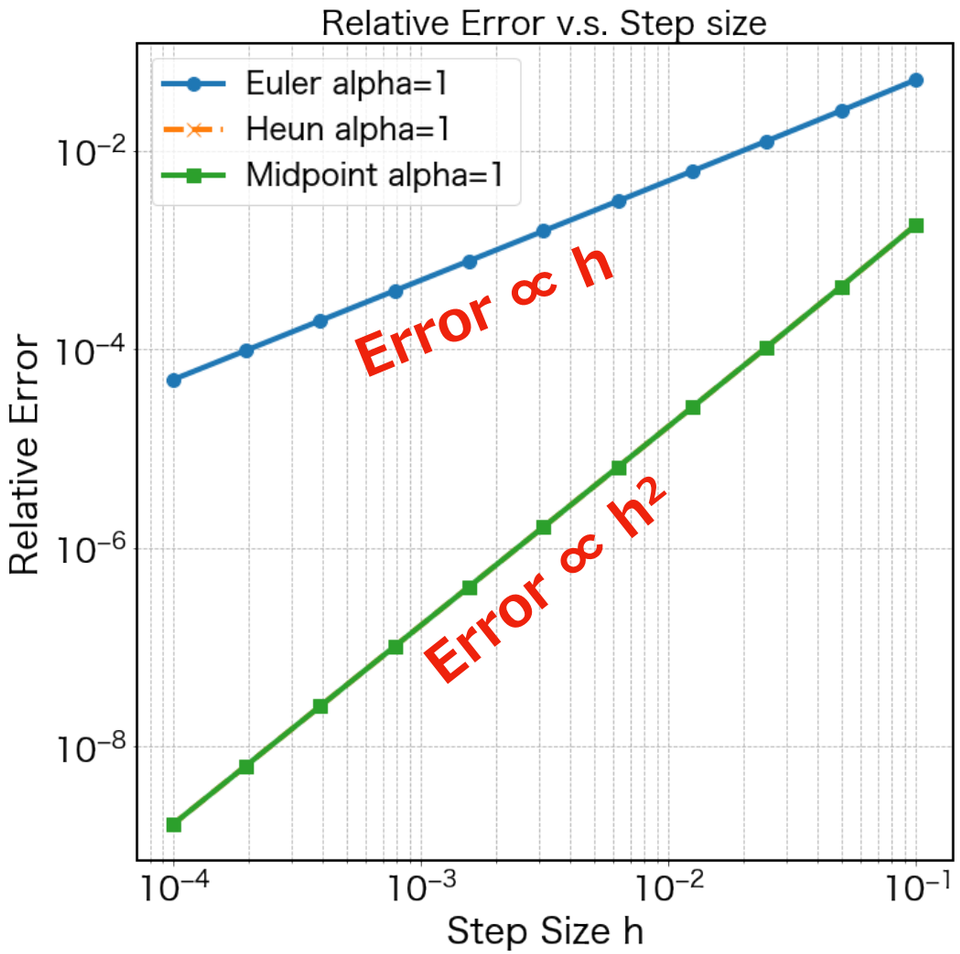
1. Calculate $f(t_n, y_n)$ from t_n and y_n.
2. Calculate $f(t_n + h, y_n + h \cdot f_n)$ from $t_n + h$, y_n, and $f(t_n, y_n)$.
3. Update y_n to y_{n+1}: $y_{n+1} = y_n + \frac{1}{2} h \left[f(t_n, y_n) + f(t_n + h, y_n + h \cdot f_n) \right]$
4. Go back to Step 1

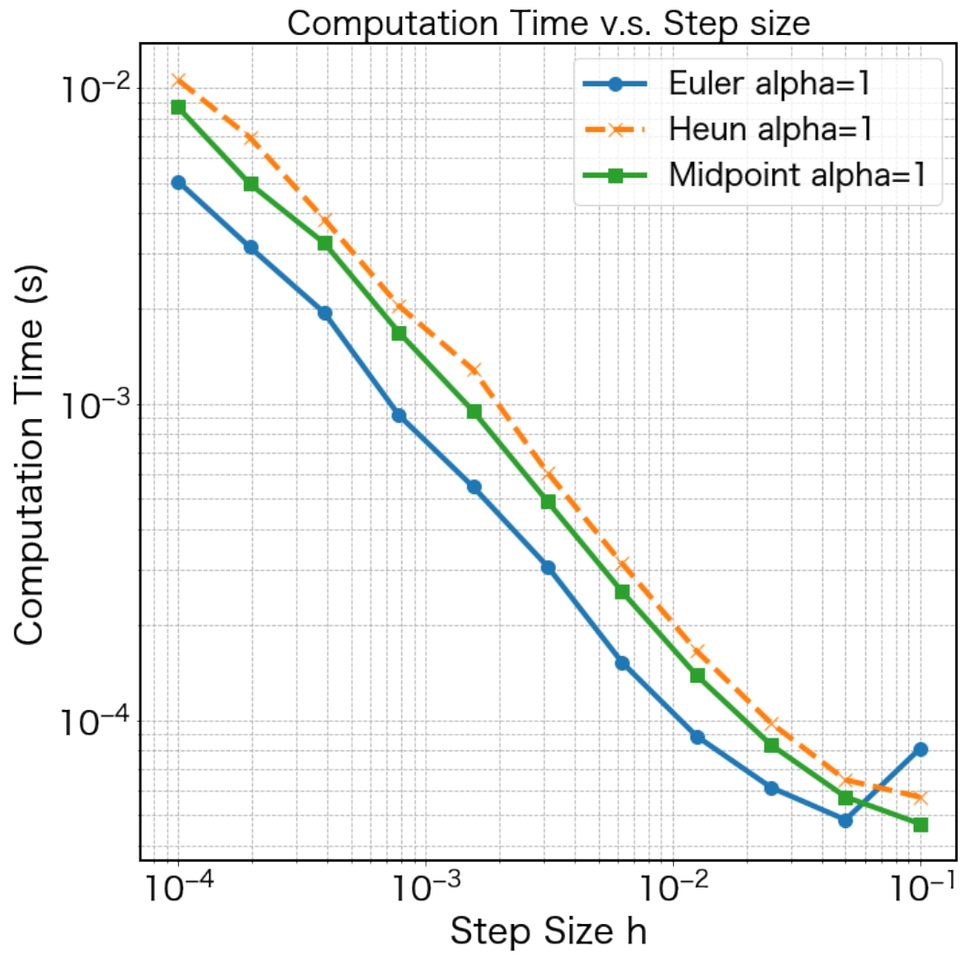
2 evaluations of the derivative f per 1 step

Modified Euler Method

1. Calculate $f(t_n, y_n)$ from t_n and y_n.
2. Calculate $f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f_n \right)$ from $t_n + \frac{h}{2}$, y_n, and $f(t_n, y_n)$.
3. Update y_n to y_{n+1}: $y_{n+1} = y_n + h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f_n \right)$
4. Go back to Step 1

Ex.02) RK2 methods





Classical Runge-Kutta (RK4) method

In the 2nd-order RK method, the coefficients and weights are chosen to ensure that the solution matches the true solution up to $O(h^2)$.

Similarly, it is possible to determine the coefficients and weights so that the approximate solution matches the true solution **up to** $O(h^4)$**.**

=> 4th-order RK method

(or **Classical RK method**)

The most general-purpose formula

(最も汎用性のある解法)

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = h \cdot f(t_n, y_n)$$

$$k_2 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right)$$

$$k_3 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right)$$

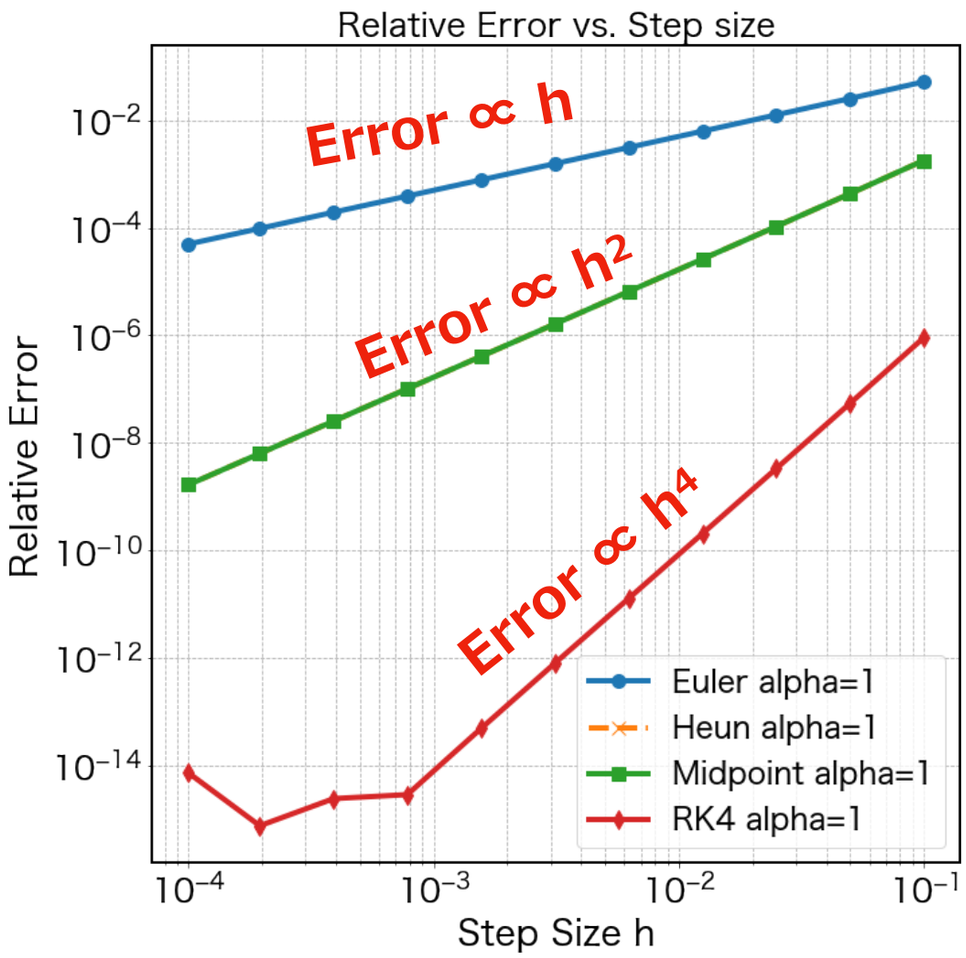
$$k_4 = h \cdot f(t_n + h, y_n + k_3)$$

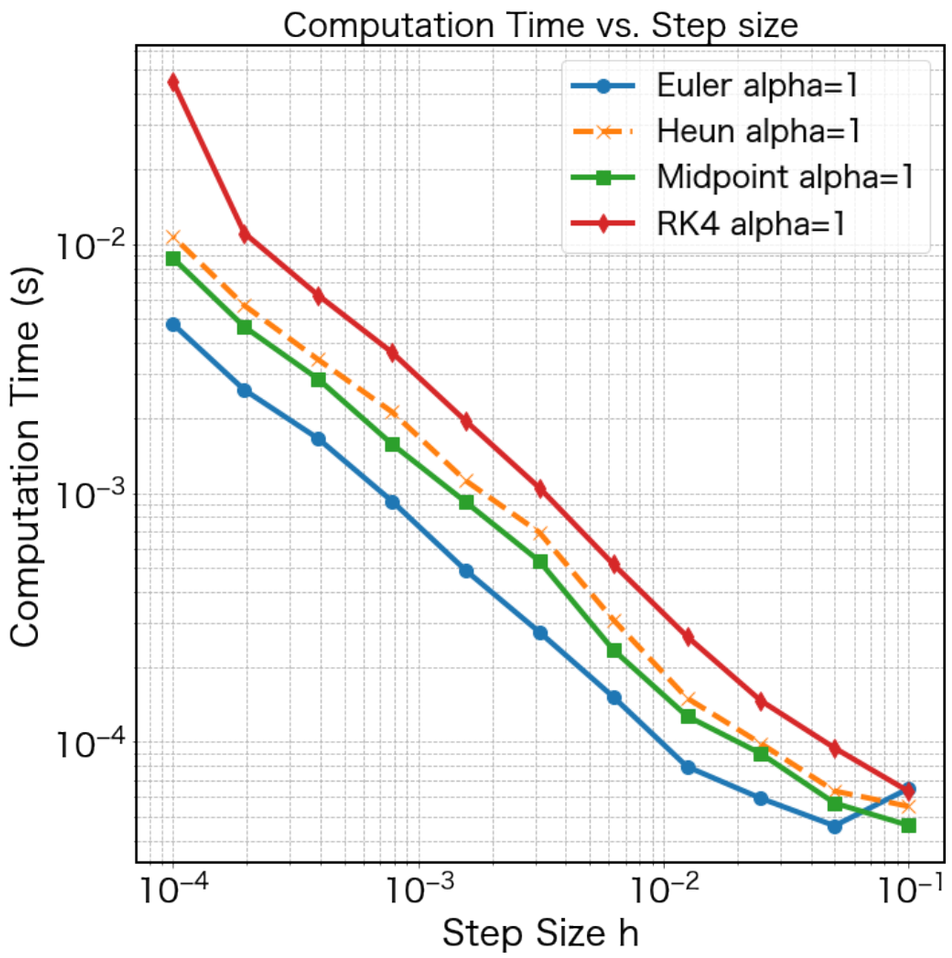
Classical RK method: Algorithm

4 evaluations of the derivative f per 1 step

1. Calculate k_1 from t_n and y_n: $k_1 = h \cdot f(t_n, y_n)$
2. Calculate k_2 from $t_n + \frac{h}{2}$, y_n and k_1: $k_2 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right)$
3. Calculate k_3 from $t_n + \frac{h}{2}$, y_n and k_2: $k_3 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right)$
4. Calculate k_4 from $t_n + h$, y_n and k_3: $k_4 = h \cdot f(t_n + h, y_n + k_3)$
5. Update y_n to y_{n+1}: $y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
6. Go back to Step 1

Ex.03) Classical RK method





Approach (2) : Multiple-step Method (多段階法)

Single-step method : determines the information at the next time step t_{n+1} using only the information from the current time step t_n

Multi-step method : determines the information at the next time step t_{n+1} using not only the current information at t_n , but also information from previous time steps $t_{n-1}, t_{n-2}, \ldots$

$$\frac{dy}{dt} = f(t, y)$$

$$\text{(discretize)} \rightarrow y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} f(t, y)dt \approx y_n + \int_{t_n}^{t_{n+1}} P_p(t, y)dt$$

Adams Methods:

discrete function $f(t, y)$ is approximated by a ***p*-th order interpolation polynomial** $P_p(t)$, and then numerical integration is performed over the interval.

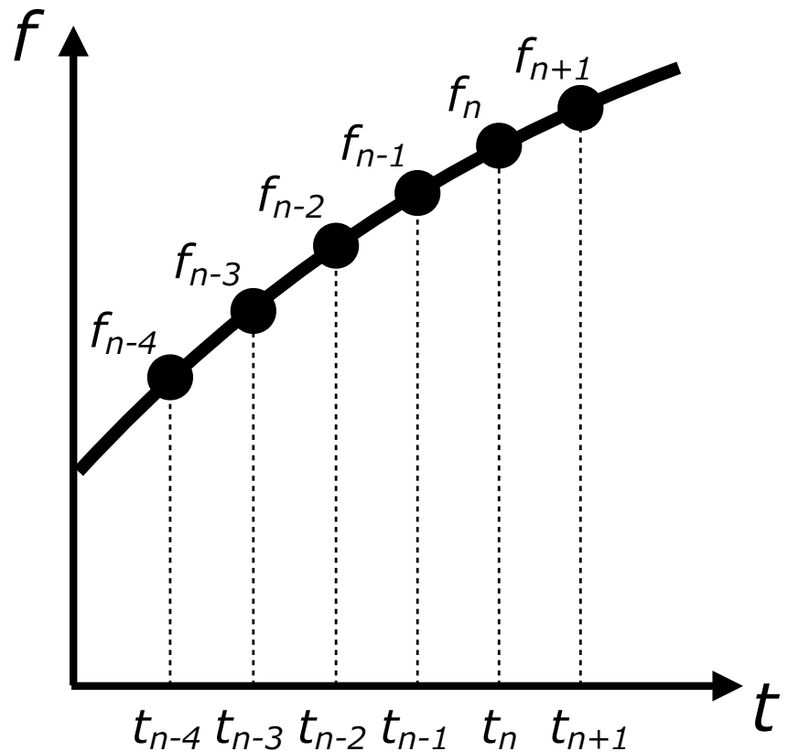
（アダムス法: 離散的な $f(t, y)$ を***p*次補間多項式** $P_p(t)$ で近似し、数値積分を行う）

Explicit/Implicit Adams methods

Constructing the Interpolation Polynomial :

1. A *p*-th order interpolation polynomial that interpolates *p+1* data points (t_{n-p}, f_{n-p}),…, (t_{n-2}, f_{n-2}), (t_{n-1}, f_{n-1}), (t_n, f_n): $P_p^{AB}(x)$
   → **Adams-Bashforth Method**
   (explicit, *p+1* step)

2. A *p*-th order interpolation polynomial that interpolates *p+1* data points (t_{n-p+1}, f_{n-p+1}),…, (t_{n-2}, f_{n-2}), (t_{n-1}, f_{n-1}), (t_n, f_n), and (t_{n+1}, f_{n+1}): $P_p^{AM}(x)$
   → **Adams-Moulton Method**
   (implicit, *p* step)



Explicit/Implicit Adams methods: Examples

1st order Adams (= Euler Methods) : $O(h)$

$y_{n+1} = y_n + hf(t_n, y_n)$ (1 step; Forward Euler 前進オイラー法)

$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$ (1 step; Backward Euler 後退オイラー法)

2nd order Adams : $O(h^2)$

$$y_{n+1} = y_n + h\left(\frac{3}{2}f(t_n, y_n) - \frac{1}{2}f(t_{n-1}, y_{n-1})\right)$$ (2 step)

$$y_{n+1} = y_n + h\left(\frac{1}{2}f(t_{n+1}, y_{n+1}) + \frac{1}{2}f(t_n, y_n)\right)$$ (1 step; Trapezoidal Formula 台形公式)

3rd order Adams : $O(h^3)$

$$y_{n+1} = y_n + h\left(\frac{23}{12}f(t_n, y_n) - \frac{16}{12}f(t_{n-1}, y_{n-1}) + \frac{5}{12}f(t_{n-2}, y_{n-2})\right)$$ (3 step)

$$y_{n+1} = y_n + h\left(\frac{5}{12}f(t_{n+1}, y_{n+1}) + \frac{8}{12}f(t_n, y_n) - \frac{1}{12}f(t_{n-1}, y_{n-1})\right)$$ (2 step)

4th order Adams : $O(h^4)$

$$y_{n+1} = y_n + h\left(\frac{55}{24}f(t_n, y_n) - \frac{59}{24}f(t_{n-1}, y_{n-1}) + \frac{37}{24}f(t_{n-2}, y_{n-2}) - \frac{9}{24}f(t_{n-3}, y_{n-3})\right)$$ (4 step)

$$y_{n+1} = y_n + h\left(\frac{9}{24}f(t_{n+1}, y_{n+1}) + \frac{19}{24}f(t_n, y_n) - \frac{5}{24}f(t_{n-1}, y_{n-1}) + \frac{1}{24}f(t_{n-2}, y_{n-2})\right)$$ (3 step)

Predictor-Corrector Methods (予測子・修正子法)

- **Explicit Adams**: not very stable because it extrapolates (外挿) and integrates the interpolation polynomial.
- **Implicit Adams**: requiring iterative calculations (反復計算).

  → To overcome the limitations of both methods, the **predictor-corrector method** is used, which combines both explicit and implicit methods to complement each other's weaknesses.

Example: 2nd order Adams PC method

Predictor (予測子)：$y_{n+1}^{(p)} = y_n + h\left(\frac{3}{2}f(t_n, y_n) - \frac{1}{2}f(t_{n-1}, y_{n-1})\right)$

Corrector (修正子)：$y_{n+1}^{(c)} = y_n + \frac{h}{2}\left(f(t_{n+1}, y_{n+1}^{(p)}) + f(t_n, y_n)\right)$

Predict ：$y_{n+1}^{(p)}$

Evaluate: $f(t_{n+1}, y_{n+1}^{(p)})$

Correct ：$y_{n+1}^{(c)}$

If the difference between $y_{n+1}^{(p)}$ and $y_{n+1}^{(c)}$ is large, $y_{n+1}^{(c)}$ is used to replace $y_{n+1}^{(p)}$, and the correction process is repeated by the corrector until the difference between $y_{n+1}^{(p)}$ and $y_{n+1}^{(c)}$ is below the required accuracy. Typically, a sufficiently small step size h is chosen so that only 1-2 corrections are necessary. ($y_{n+1}^{(p)}$ と $y_{n+1}^{(c)}$ が十分な精度で一致するまで、$y_{n+1}^{(c)}$と$y_{n+1}^{(p)}$として使い反復計算する必要があるが、通常は1-2回の反復計算回数に留める) i.e. **PEC** or **P(EC)²**

Other 2nd-order Predictor-Corrector Methods

2nd order Adams method

$$y_{n+1}^{(p)} = y_n + h\left(\frac{3}{2}f(t_n, y_n) - \frac{1}{2}f(t_{n-1}, y_{n-1})\right)$$

$$y_{n+1}^{(c)} = y_n + \frac{h}{2}\left(+f(t_n, y_n) + f(t_{n+1}, y_{n+1}^{(p)})\right)$$: Trapezoidal method

Heun method (2nd order RK)

$$y_{n+1}^{(p)} = y_n + h \cdot f(t_n, y_n)$$: Forward Euler method

$$y_{n+1}^{(c)} = y_n + \frac{h}{2}\left(f(t_n, y_n) + f(t_{n+1}, y_{n+1}^{(p)})\right)$$: Trapezoidal method

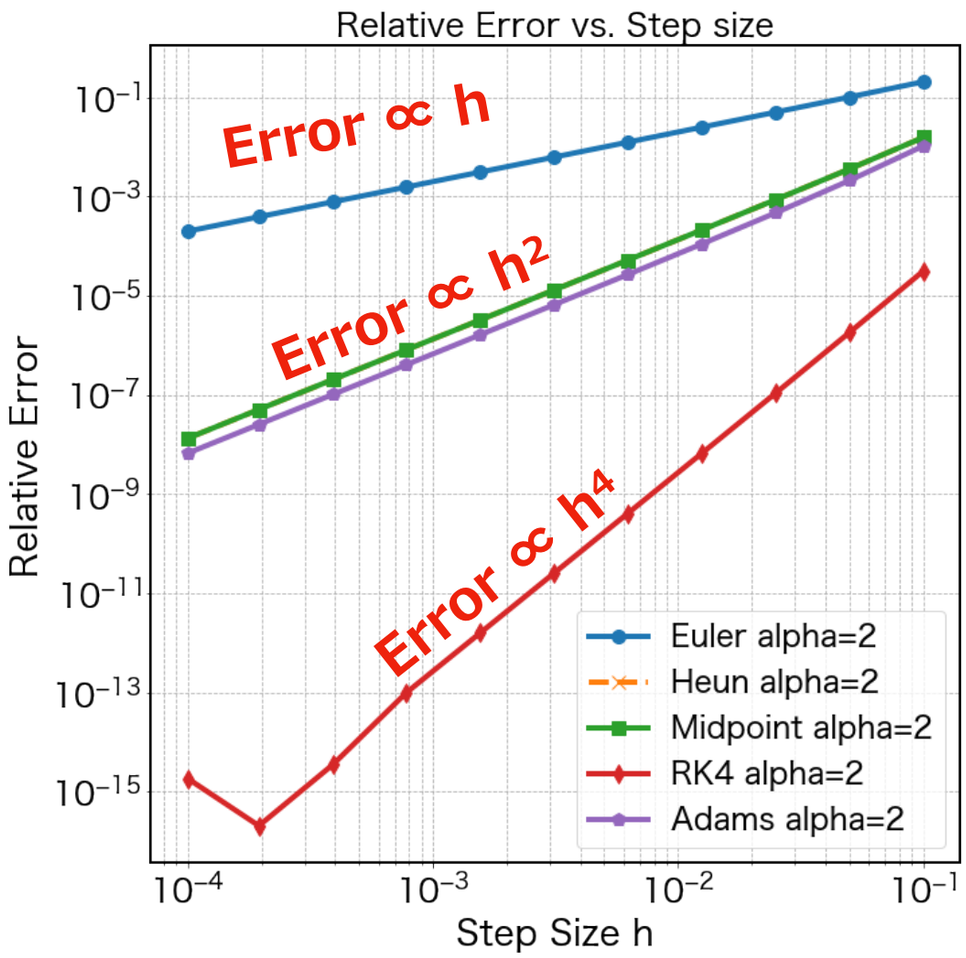
Modified Euler method (2nd order RK)

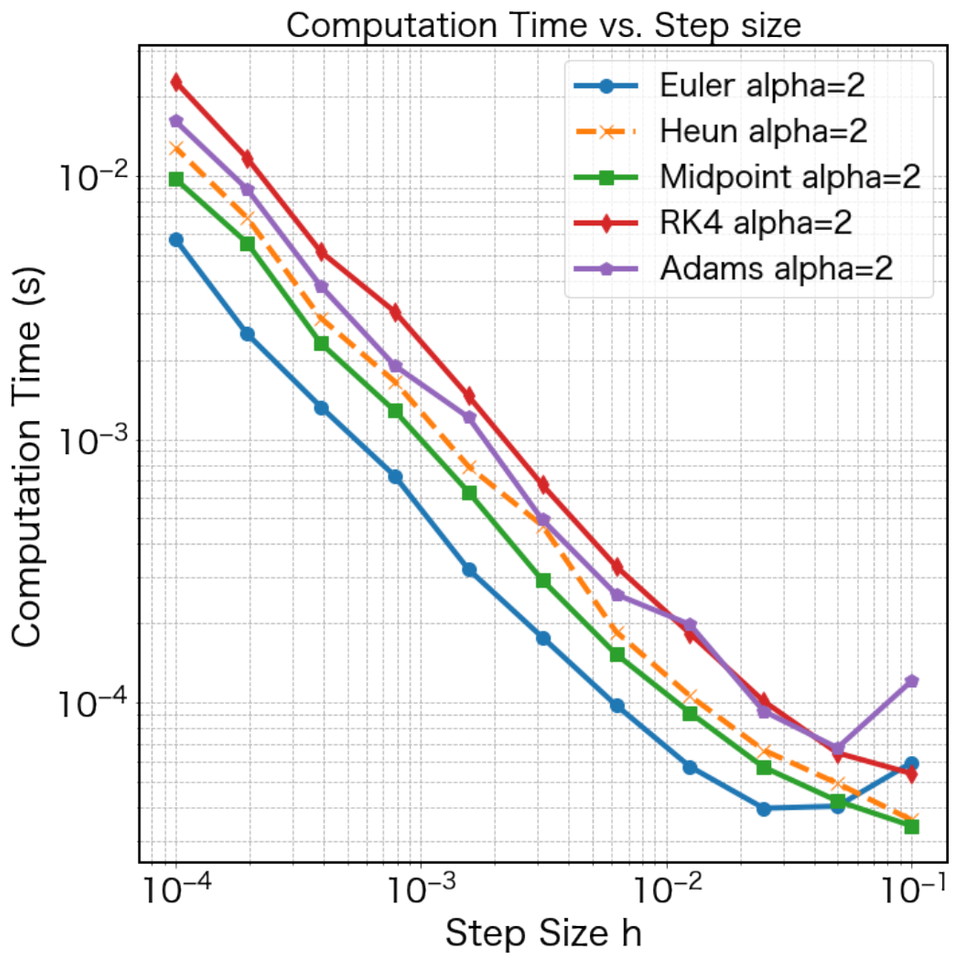
$$y_{n+1/2}^{(p)} = y_n + \frac{h}{2}f(t_n, y_n)$$: Forward Euler method

$$y_{n+1}^{(c)} = y_n + h \cdot f\left(t_n + \frac{h}{2}, y_{n+1/2}^{(p)}\right)$$: Midpoint method

Since these formulas are identical to RK2 methods, $O(h^2)$ accuracy is ensured with just one correction step.
(RK2公式になっているため、
1回の修正子で$O(h^2)$精度となる)

Ex.04) 2nd Adams PEC Method





RK4 vs. 4th Adams

RK4 method

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = h \cdot f(t_n, y_n)$$

$$k_2 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right)$$

$$k_3 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right)$$

$$k_4 = h \cdot f(t_n + h, y_n + k_3)$$

4 evaluations of the derivative are required to perform one step update.

4th Adams PC method

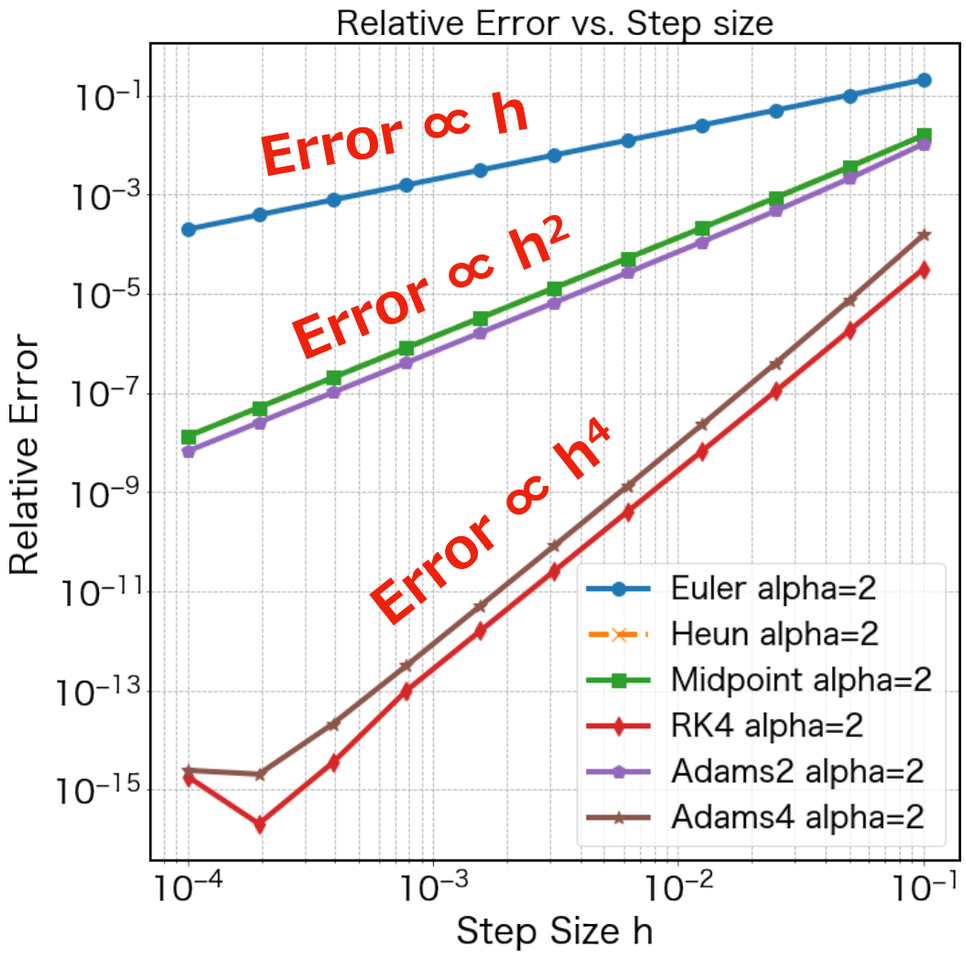
$$y_{n+1}^{(p)} = y_n + \frac{h}{24}\left(55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})\right)$$

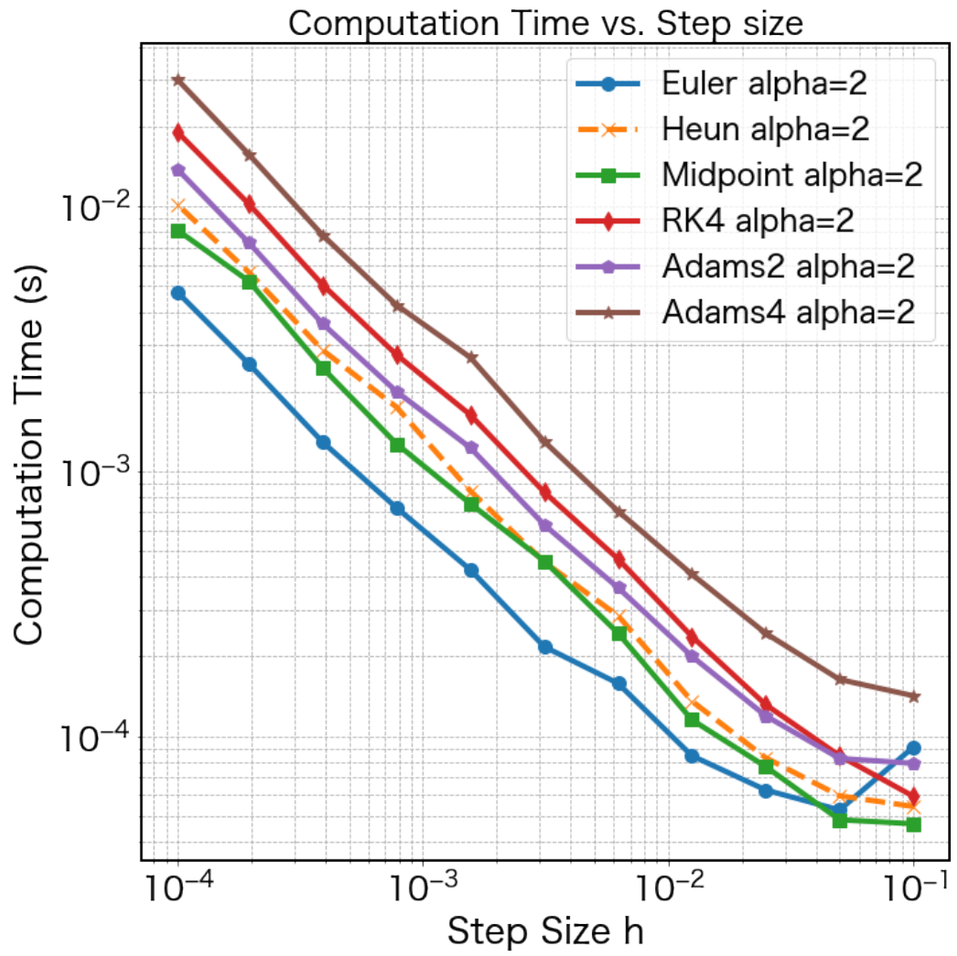
$$y_{n+1}^{(c)} = y_n + \frac{h}{24}\left(9f(t_{n+1}, y_{n+1}^{(p)}) + 19f(t_n, y_n) - 5f(t_{n-1}, y_{n-1}) + f(t_{n-2}, y_{n-2})\right)$$

2 evaluations for the derivative are required to perform the step update.

However, if iterative calculations are necessary for the predictor-corrector step, the number of derivative evaluations increases.

Ex.04) 4th Adams PEC Method





4th order RK and Adams Mthods

Feature	4th Runge-Kutta (RK4)	4th Adams-Bashforth/Moulton
Step **(段数)**	Single-step (Explicit)	4 step (Explicit and Implicit)
Accuracy **(精度)**	O(h^4)	O(h^4)
Function Evaluations per Step **(1ステップあたりの導関数の評価回数)**	**4 evaluations/step**	**2-3 evaluations/step**
Memory Requirements **(必要なメモリ量)**	Low (only current step used)	High (stores multiple previous steps)
Stability **(安定性)**	Good for non-stiff problems	Explicit: Less stable; Implicit: More stable (better for stiff problems)
Complexity **(実装の複雑さ)**	**Simple** to implement	More **complex**, especially for implicit methods
Step Size Variability **(ステップ幅の可変性)**	Easily adaptable to **variable step** sizes	Typically uses **fixed step** sizes
Use Cases **(利用場面)**	General-purpose, short/medium simulations	Long simulations, stiff equations (implicit)

Summary: Numerical Methods for Solving ODEs

- What is **Computational Astrophysics**?
  - Recreating and observing natural phenomena within a computer to understand astronomical phenomena. (自然現象を計算機の中に再現・観測し、天文現象を理解する)
- **Finite Difference Methods (有限差分法)**
  - Explicit / Implicit Methods (陽的 / 陰的)
  - Single-Step / Multistep Methods (一段解法 / 多段階法)
  - Local Truncation Error / Global Truncation Error (局所誤差 / 累積誤差)
  - Euler method: Global $O(h)$ error → Low accuracy, “not” practical for applications
- Approach (1): **Runge-Kutta Methods (ルンゲ・クッタ法)**
  - Heun method / Modified Euler method: Global $O(h^2)$ error
  - **Classical RK (RK4):** Global $O(h^4)$ error, **the most general-purpose method**
  - Dormand-Prince formula: Global $O(h^5)$ error, with automatic step size adjustment
- Approach (2): Multistep methods, **Predictor-Corrector Methods (予測子・修正子法)**
  - Adams methods: Global $O(h^p)$ error
  - **Predictor-Corrector integrator**, e.g. **PEC**, **P(EC)^2**

Lecture #2 (2024/11/1)

Numerical Methods for N-body Systems 1: Time Integrations

(重力多体系の時間積分法)

1. Gravitational Many-body sytems
2. Relaxation, Collisionless/Collisional Systems
3. Leap-frog Method (リープフロッグ公式)
4. Hermite Method (エルミート公式)

Recommend: **Dehnen & Read (2011)**, *The European Physical Journal Plus*, Volume 126, article id. 55
This review paper provides a well-organized summary of *N*-body simulation methods for both collisionless and collisional systems. For those who wish to gain a deeper understanding, I recommend referring to it.

Eur. Phys. J. Plus (2011) **126**: 55
DOI 10.1140/epjp/i2011-11055-3

THE EUROPEAN PHYSICAL JOURNAL PLUS

Regular Article

N-body simulations of gravitational dynamics

W. Dehnen[1,a] and J.I. Read[1,2]

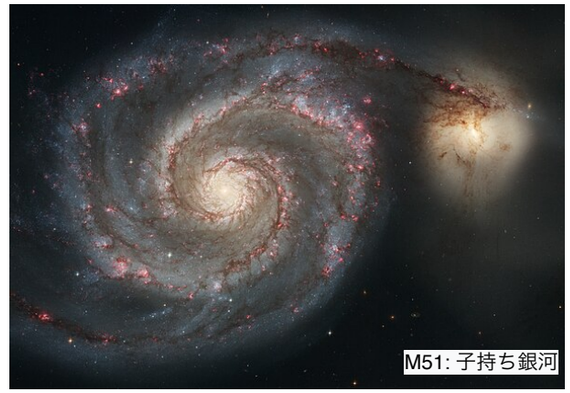
[1] Department of Physics and Astronomy, University of Leicester - Leicester LE17RH, UK
[2] Institute for Astronomy, Department of Physics, ETH Zürich - Wolfgang-Pauli-Strasse 16, CH-8093 Zürich, Switzerland

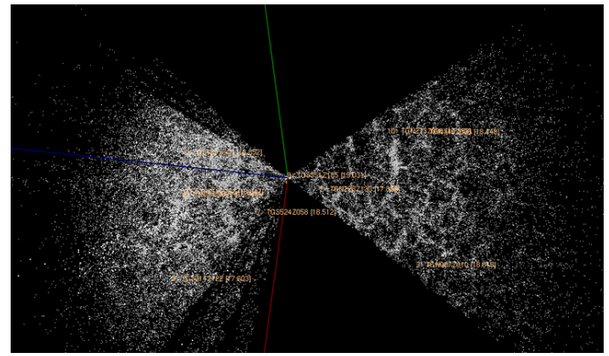
Received: 14 April 2011
Published online: 30 May 2011 – © Società Italiana di Fisica / Springer-Verlag 2011

Abstract. We describe the astrophysical and numerical basis of *N*-body simulations, both of collisional stellar systems (dense star clusters and galactic centres) and collisionless stellar dynamics (galaxies and large-scale structure). We explain and discuss the state-of-the-art algorithms used for these quite different regimes, attempt to give a fair critique, and point out possible directions of future improvement and development. We briefly touch upon the history of *N*-body simulations and their most important results.

Examples

- Solar system, planetary system (planets)
- Planetary disk (planetesimals)
- Open cluster (stars)
- Globular cluster (stars)
- Galaxy (stars)
- Group of galaxies (galaxies)
- Cluster of galaxies (galaxies)
- Cosmic Large-Scale Structures (galaxies)





Gravitational Many-Body Systems

Equations of Motion for Many-Body System:

$$m_i \frac{d^2\mathbf{x}_i}{dt^2} = \sum_{j=1, j\neq i}^{N} \mathbf{f}_{i,j}$$

x_i : position mass of i-th particle

m_i : mass of i-th particle

f_{ij}: mutual force exerted by j-th particle on i-th particle

In a gravitational Many-body system,

$$\mathbf{f}_{i,j} = G m_i m_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}$$

Newtonian gravity

G: the gravitational constant

4.3×10^{-6} kpc (km/s)^2/M$_\odot$

Thus,

$$m_i \frac{d^2\mathbf{x}_i}{dt^2} = \sum_{j=1, j\neq i}^{N} G m_i m_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} \quad ==> \quad \boxed{\frac{d^2\mathbf{x}_i}{dt^2} = \sum_{j=1, j\neq i}^{N} G m_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}}$$

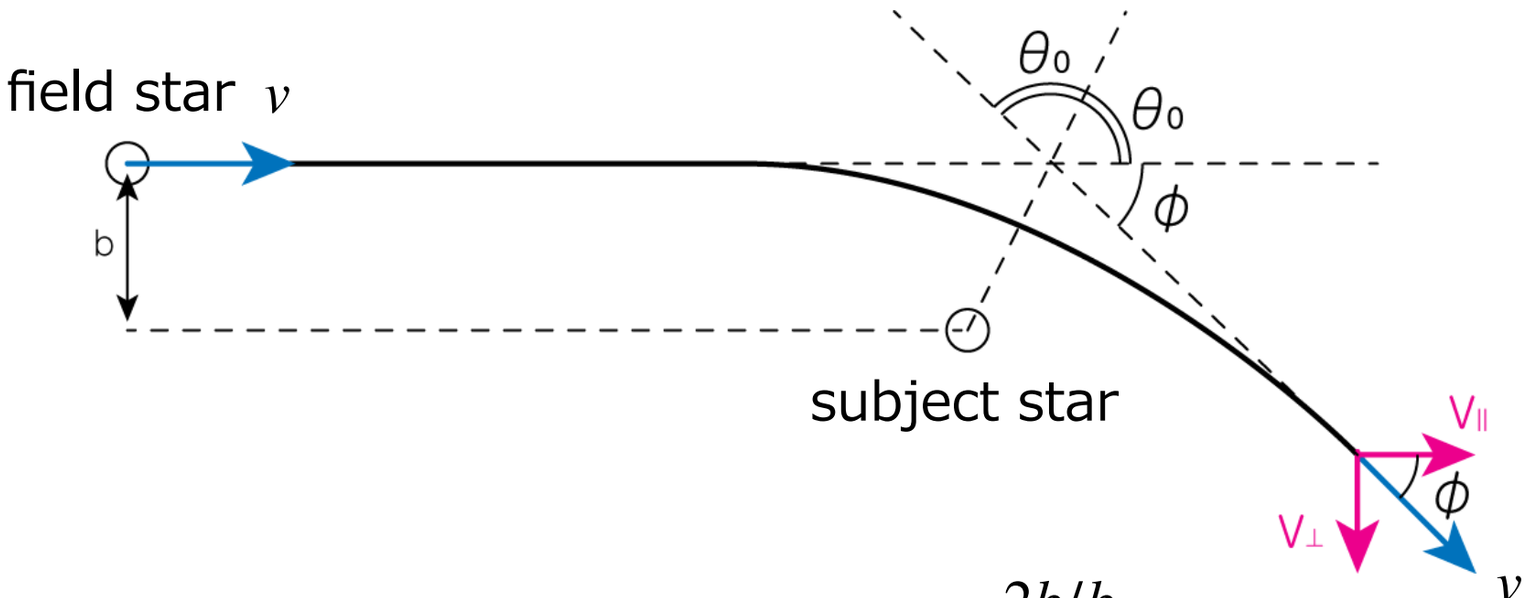
3N dim. 2nd-order ODEs

(3N次元連立2階常微分方程式)

Two-body Scattering (two-body problem)

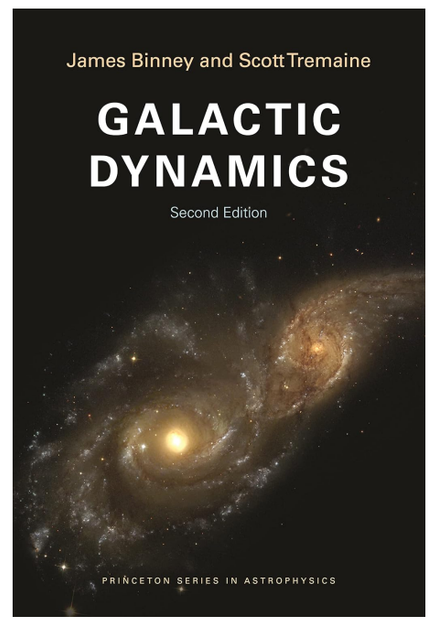
Difference from smooth average potential due to finite number of stars.

A field star approaches the subject star at speed v and impact parameter b.



Bible!

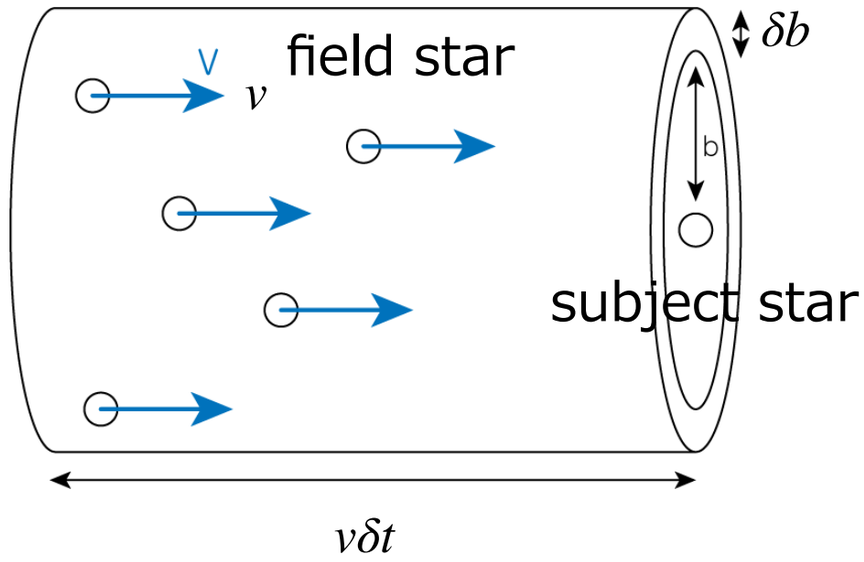
Binney & Tremaine (2008)



Velocity change: $\Delta v_{\perp} = v \sin\phi = v \dfrac{2b/b_{90}}{1+(b/b_{90})^2}$

where $b_{90} \equiv \dfrac{Gm}{v^2}$: impact parameter for 90-degree deflection

Two-body Relaxation Time



The number of encountering stars per δt:

$$\frac{dN}{dt} = 2\pi b \delta b \times n \times v$$

Cumulative velocity changes:

$$\langle \Delta v_\perp^2 \rangle = \sum \Delta v_\perp^2 \frac{dN}{dt} = \int_{b_{90}}^{b_{\max}} \Delta v_\perp^2 2\pi v n b \, \mathrm{d}b \sim \frac{G^2 n m^2 \ln \Lambda}{v}$$

Coulomb logarithm: $\ln \Lambda \equiv \ln\left(\frac{b_{\max}}{b_{90}}\right) \sim \left(\frac{R}{Gm/v^2}\right)$, where R: system size

Relaxation time: $t_{\mathrm{relax}} \equiv \frac{v^2}{\langle \Delta v_\perp^2 \rangle} \sim \frac{v^3}{G^2 n m^2 \ln \Lambda} \sim 0.1 \frac{N}{\ln N} t_{\mathrm{cross}}$ (virialized $v^2 \sim \frac{GNm}{R}$)

2体緩和時間

Crossing time : $t_{\mathrm{cross}} = \frac{R}{v}$

横断時間

if $t_{\mathrm{relax}}/t_{\mathrm{cross}} \gg 1$, the star moves in a smooth orbit under the influence of the average gravitational field. i.e. negligible effect of gravitational scattering.

Collisionless/Collisional Systems

Collisionless systems (無衝突系) ← **Leap-frog integrator** $O(h^2)$

- negligible two-body relaxation, ***age/t_{relax} << 1***
- e.g. galaxies

Collisional systems (衝突系) ← **Hermite integrator** $O(h^4)$

- evolution due to two-body scattering, ***age/t_{relax} >~ 1***
- e.g. star clusters, galaxy clusters

	N	t_{cross} **(yr)**	t_{relax} **(yr)**	**age (yr)**	**age/t_{relax}**
open cluster	~100	~10^6	10^7	~10^7	<~1
globular cluster	~10^5	~10^5	10^9	<~10^{10}	<~10
galaxy	~10^{11}	~10^8	10^{17}	10^{10}	~10^{-7}
galaxy cluster	~10^3	~10^9	10^{10}	<~10^{10}	<~1

N-Body Simulations

Equations of Motion:

$$\frac{d^2\mathbf{x}_i}{dt^2} = \sum_{j=1,j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}$$

3N dim. 2nd-order ODEs
(3N次元連立2階常微分方程式)

==>

Converting to
1st-Order ODEs

$$\begin{cases} \dfrac{d\mathbf{v}_i}{dt} = \displaystyle\sum_{j=1,j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} \\ \dfrac{d\mathbf{x}_i}{dt} = \mathbf{v}_i \end{cases}$$

6N dim. simultaneous 1st-order ODEs
(6N次元連立1階常微分方程式)

We can apply the numerical methods for ODEs learned in the previous lecture!

Difficulties

- LHS: Accurate time integration to treat **wide dynamical timescale**
- RHS: Large calc. cost, $O(N^2)$, due to **gravitational interactions**

Example: Euler Method

Eq. of Motion (EoM):

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i$$

$$\frac{d\mathbf{v}_i}{dt} = \sum_{j=1, j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} = \mathbf{a}_i(\mathbf{x})$$

ODE: $\frac{dy}{dt} = f(t, y)$

Euler Methods

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

EoM w/ Euler Method

$$\mathbf{x}_{i,n+1} = \mathbf{x}_{i,n} + \Delta t \mathbf{v}_{i,n}$$

$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n} + \Delta t \mathbf{a}_{i,n}$$

Example: RK2 Heun Method

Eq. of Motion (EoM):

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i$$

$$\frac{d\mathbf{v}_i}{dt} = \sum_{j=1, j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} = \mathbf{a}_i(\mathbf{x})$$

ODE: $\frac{dy}{dt} = f(t, y)$

Heun Method:

$$y_{n+1} = y_n + \frac{1}{2}h\left[f(t_n, y_n) + f(t_n + h, y_n + h \cdot f_n)\right]$$

EoM w/ Heum Method

$$\mathbf{x}_{i,n+1} = \mathbf{x}_{i,n} + \frac{\Delta t}{2}\left[\mathbf{v}_{i,n} + (\mathbf{v}_{i,n} + \Delta t \mathbf{a}_i(\mathbf{x}_{i,n}))\right]$$

$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n} + \frac{\Delta t}{2}\left[\mathbf{a}_i(\mathbf{x}_{i,n}) + \mathbf{a}_i(\mathbf{x}_{i,n} + \Delta t \mathbf{v}_{i,n})\right]$$

Ex.05) Kepler Problem

```python
# Define the Forward Euler Method Function
def euler_method(r0, v0, t0, t_end, h):
    """
    Solve the Kepler problem using the explicit Forward Euler method.

    Parameters:
        r0 (np.ndarray): Initial position vector [x0, y0]
        v0 (np.ndarray): Initial velocity vector [vx0, vy0]
        t0 (float): Initial time
        t_end (float): End time
        h (float): Step size

    Returns:
        t_values (np.ndarray): Array of time points
        r_values (np.ndarray): Array of position vectors at each time point
        v_values (np.ndarray): Array of velocity vectors at each time point
        energy_errors (np.ndarray): Array of energy errors at each time point
    """
    N = int(round((t_end - t0) / h))  # Number of steps
    t_values = np.linspace(t0, t_end, N + 1)

    r_values = np.zeros((N + 1, 2))  # Store position vectors [x, y]
    v_values = np.zeros((N + 1, 2))  # Store velocity vectors [vx, vy]

    r_values[0] = r0
    v_values[0] = v0

    # Calculate initial energy
    initial_energy = total_energy(r0, v0)
    energy_errors = np.zeros(N + 1)
    energy_errors[0] = 0  # Initial energy error is 0

    for n in range(N):
        r_n = r_values[n]
        v_n = v_values[n]

        # Euler method: update position and velocity
        r_values[n + 1] = r_n + h * v_n  # Update position
        v_values[n + 1] = v_n + h * gravity_acceleration(r_n)  # Update velocity

        # Calculate energy at this step
        energy_n = total_energy(r_values[n + 1], v_values[n + 1])
        energy_errors[n + 1] = abs(energy_n - initial_energy)  # Absolute error in energy

    return t_values, r_values, v_values, energy_errors
```

Euler Method

$$\mathbf{x}_{i,n+1} = \mathbf{x}_{i,n} + \Delta t \mathbf{v}_{i,n}$$

$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n} + \Delta t \mathbf{a}_{i,n}$$

```python
def rk2_heun_method(r0, v0, t0, t_end, h):
    """
    Solve the Kepler problem using Heun's method (improved Euler method).

    Parameters:
        r0 (np.ndarray): Initial position vector [x0, y0]
        v0 (np.ndarray): Initial velocity vector [vx0, vy0]
        t0 (float): Initial time
        t_end (float): End time
        h (float): Step size

    Returns:
        t_values (np.ndarray): Array of time points
        r_values (np.ndarray): Array of position vectors at each time point
        v_values (np.ndarray): Array of velocity vectors at each time point
        energy_errors (np.ndarray): Array of energy errors at each time point
    """
    N = int(round((t_end - t0) / h))  # Number of steps
    t_values = np.linspace(t0, t_end, N + 1)

    r_values = np.zeros((N + 1, 2))  # Store position vectors [x, y]
    v_values = np.zeros((N + 1, 2))  #

    r_values[0] = r0
    v_values[0] = v0

    # Calculate initial energy
    initial_energy = total_energy(r0,
    energy_errors = np.zeros(N + 1)
    energy_errors[0] = 0  # Initial en

    for n in range(N):
        r_n = r_values[n]
        v_n = v_values[n]
        a_n = gravity_acceleration(r_n)

        # Predictor step: Forward Euler for both r and v
        r_predictor = r_n + h * v_n
        v_predictor = v_n + h * a_n

        # Corrector step: using average of current and predicted values
        r_values[n + 1] = r_n + (h / 2) * (v_n + v_predictor)
        v_values[n + 1] = v_n + (h / 2) * (a_n + gravity_acceleration(r_predictor))

        # Calculate energy at this step
        energy_n = total_energy(r_values[n + 1], v_values[n + 1])
        energy_errors[n + 1] = abs(energy_n - initial_energy)  # Absolute error in energy

    return t_values, r_values, v_values, energy_errors
```

Heun Method

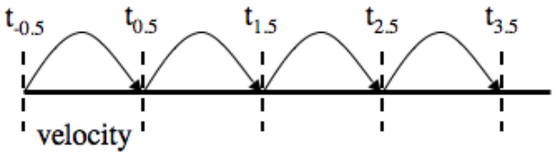
$$\mathbf{x}_{i,n+1} = \mathbf{x}_{i,n} + \frac{\Delta t}{2}\left[\mathbf{v}_{i,n} + (\mathbf{v}_{i,n} + \Delta t \mathbf{a}_i(\mathbf{x}_{i,n}))\right]$$

$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n} + \frac{\Delta t}{2}\left[\mathbf{a}_i(\mathbf{x}_{i,n}) + \mathbf{a}_i(\mathbf{x}_{i,n} + \Delta t \mathbf{v}_{i,n})\right]$$

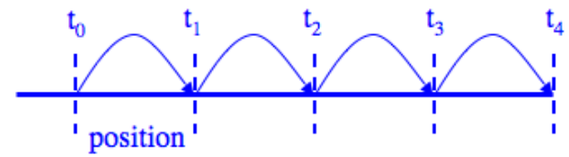
Leapfrog Integrator: Formula

$$\mathbf{v}_{n+1/2} = \mathbf{v}_n + \mathbf{a}_n \frac{h}{2}$$
$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1/2} h$$
$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \mathbf{a}_{n+1} \frac{h}{2}$$

$$\mathbf{v}_{n+1/2} = \mathbf{v}_{n-1/2} + \mathbf{a}_n h$$



$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1/2} h$$



Update $\mathbf{x}$ and $\mathbf{v}$ at times offset by $h/2$
(leapfrogging each other in updates).
1/2hステップずれた時刻でお互いを飛び越えてx, vを更新

- Global $O(h^2)$ error, i.e. **2nd order formula**
- One-step method ($n \to n+1$)
- Autostart (自己開始公式)
- Symplectic (energy is well conserved)

Leapfrog Integrator: Derivation

Taylor expand exact solutions around $t = t_n$:

$$\mathbf{x}(t_n + h) = \mathbf{x}(t_n) + \dot{\mathbf{x}}(t_n)h + \frac{1}{2}\ddot{\mathbf{x}}(t_n)h^2 + O(h^3) = \mathbf{x}(t_n) + \mathbf{v}(t_n)h + \frac{1}{2}\mathbf{a}(t_n)h^2 + O(h^3)$$

$$\mathbf{v}(t_n + h) = \mathbf{v}(t_n) + \dot{\mathbf{v}}(t_n)h + \frac{1}{2}\ddot{\mathbf{v}}(t_n)h^2 + O(h^3) = \mathbf{v}(t_n) + \mathbf{a}(t_n)h + \frac{1}{2}\dot{\mathbf{a}}(t_n)h^2 + O(h^3)$$

$$\mathbf{a}(t_n + h) = \mathbf{a}(t_n) + \dot{\mathbf{a}}(t_n)h + \frac{1}{2}\ddot{\mathbf{a}}(t_n)h^2 + O(h^3)$$

Approximate solutions with local truncation errors of $O(h^3)$:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n h + \frac{1}{2}\mathbf{a}_n h^2 = \mathbf{x}_n + \left(\mathbf{v}_n + \mathbf{a}_n \frac{h}{2}\right) h = \mathbf{x}_n + \mathbf{v}_{n+1/2} h$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n h + \frac{1}{2}\dot{\mathbf{a}}_n h^2 = \left(\mathbf{v}_n + \mathbf{a}_n \frac{h}{2}\right) + \left(\mathbf{a}_n + \dot{\mathbf{a}}_n h\right)\frac{h}{2} = \mathbf{v}_{n+1/2} + \left(\mathbf{a}_n + \dot{\mathbf{a}}_n h\right)\frac{h}{2} = \mathbf{v}_{n+1/2} + \mathbf{a}_{n+1}\frac{h}{2} + \cancel{O(h^3)}$$

where $\mathbf{v}_{n+1/2} = \mathbf{v}_n + \mathbf{a}_n \frac{h}{2}$ and $\mathbf{a}_{n+1} = \mathbf{a}_n + \dot{\mathbf{a}}_n h = \mathbf{a}(t_n + h) + O(h^2)$

Leapfrog Integrator: Algorithm

1. Calculate $\mathbf{v}_{n+1/2}$ from $\mathbf{v}_n$ and $\mathbf{a}_n$: $\mathbf{v}_{n+1/2} = \mathbf{v}_n + \mathbf{a}_n \frac{h}{2}$ (**Kick**)
2. Calculate $\mathbf{x}_{n+1}$ from $\mathbf{x}_n$ and $\mathbf{v}_{n+1/2}$: $\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1/2} h$ (**Drift**)
3. Calculate $\mathbf{a}_{n+1}$ from $\mathbf{x}_{n+1}$ (**gravity calculation**)
4. Calculate $\mathbf{v}_{n+1}$ from $\mathbf{v}_{n+1/2}$ and $\mathbf{a}_{n+1}$: $\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \mathbf{a}_{n+1} \frac{h}{2}$ (**Kick**)
5. Go back to Step 1

1 evaluation of the gravity per 1 step

RK2 vs. Leapfrog Methods

Leapfrog method has the same $O(h^2)$ accuracy as RK2 method,
but requires **only one derivative (gravity) evaluation**, making it **faster**.

RK2 Method (Heun Method)

1. Calculate $\mathbf{a}(\mathbf{x}_n)$ from $\mathbf{x}_n$
2. Predict $\mathbf{x}^p_{n+1} = \mathbf{x}_n + h\mathbf{v}_n$
3. Calculate $\mathbf{a}(\mathbf{x}^p_{n+1})$ from $\mathbf{x}^p_{n+1}$
4. Update $\mathbf{x_n}$ and $\mathbf{v_n}$ to $\mathbf{x}_{n+1}$ and $\mathbf{v}_{n+1}$
5. Go back to Step 1

2 evaluations of the derivative f (gravity) per 1 step

Leapfrog Method

1. Calculate $\mathbf{v}_{n+1/2}$ from $\mathbf{v}_n$ and $\mathbf{a}_n$
2. Calculate $\mathbf{x}_{n+1}$ from $\mathbf{x}_n$ and $\mathbf{v}_{n+1/2}$
3. Calculate $\mathbf{a}_{n+1}$ from $\mathbf{x}_{n+1}$
4. Calculate $\mathbf{v}_{n+1}$ from $\mathbf{v}_{n+1/2}$ and $\mathbf{a}_{n+1}$
5. Go back to Step 1

1 evaluation of the gravity per 1 step

Ex.05) Kepler Problem

```python
def rk2_heun_method(r0, v0, t0, t_end, h):
    """
    Solve the Kepler problem using Heun's method (improved Euler method).

    Parameters:
        r0 (np.ndarray): Initial position vector [x0, y0]
        v0 (np.ndarray): Initial velocity vector [vx0, vy0]
        t0 (float): Initial time
        t_end (float): End time
        h (float): Step size

    Returns:
        t_values (np.ndarray): Array of time points
        r_values (np.ndarray): Array of position vectors at each time point
        v_values (np.ndarray): Array of velocity vectors at each time point
        energy_errors (np.ndarray): Array of energy errors at each time point
    """
    N = int(round((t_end - t0) / h))  # Number of steps
    t_values = np.linspace(t0, t_end, N + 1)

    r_values = np.zeros((N + 1, 2))  # Store position vectors [x, y]
    v_values = np.zeros((N + 1, 2))

    r_values[0] = r0
    v_values[0] = v0

    # Calculate initial energy
    initial_energy = total_energy(r0
    energy_errors = np.zeros(N + 1)
    energy_errors[0] = 0  # Initial

    for n in range(N):
        r_n = r_values[n]
        v_n = v_values[n]
        a_n = gravity_acceleration(r_n)

        # Predictor step: Forward Euler for both r and v
        r_predictor = r_n + h * v_n
        v_predictor = v_n + h * a_n

        # Corrector step: using average of current and predicted values
        r_values[n + 1] = r_n + (h / 2) * (v_n + v_predictor)
        v_values[n + 1] = v_n + (h / 2) * (a_n + gravity_acceleration(r_predictor))

        # Calculate energy at this step
        energy_n = total_energy(r_values[n + 1], v_values[n + 1])
        energy_errors[n + 1] = abs(energy_n - initial_energy)  # Absolute error in energy

    return t_values, r_values, v_values, energy_errors
```

Heun Method

$$\mathbf{x}_{i,n+1} = \mathbf{x}_{i,n} + \frac{\Delta t}{2}\left[\mathbf{v}_{i,n} + (\mathbf{v}_{i,n} + \Delta t \mathbf{a}_i(\mathbf{x}_{i,n}))\right]$$

$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n} + \frac{\Delta t}{2}\left[\mathbf{a}_i(\mathbf{x}_{i,n}) + \mathbf{a}_i(\mathbf{x}_{i,n} + \Delta t \mathbf{v}_{i,n})\right]$$

```python
def leapfrog_method(r0, v0, t0, t_end, h):
    """
    Solve the Kepler problem using the Leapfrog method.

    Parameters:
        r0 (np.ndarray): Initial position vector [x0, y0]
        v0 (np.ndarray): Initial velocity vector [vx0, vy0]
        t0 (float): Initial time
        t_end (float): End time
        h (float): Step size

    Returns:
        t_values (np.ndarray): Array of time points
        r_values (np.ndarray): Array of position vectors at each time point
        v_values (np.ndarray): Array of velocity vectors at each time point
        energy_errors (np.ndarray): Array of energy errors at each time point
    """
    N = int(round((t_end - t0) / h))  # Number of steps
    t_values = np.linspace(t0, t_end, N + 1)

    r_values = np.zeros((N + 1, 2))  # Store position vectors [x, y]
    v_values = np.zeros((N + 1, 2))  # Store velocity vectors [vx, vy]
    a_values = np.zeros((N + 1, 2))  # Store velocity vectors [ax, ay]

    r_values[0] = r0
    v_values[0] = v0
    a_values[0] = gravity_acceleration(r0)

    # Calculate initial energy
    initial_energy = total_energy(r0, v0)
    energy_errors = np.zeros(N + 1)
    energy_errors[0] = 0  # Initial energy error is 0

    for n in range(N):
        # kick
        v_n = v_values[n]
        v_half = v_n + 0.5 * h * a_values[n]

        # drift
        r_n = r_values[n]
        r_values[n + 1] = r_n + h * v_half

        # kick
        a_values[n + 1] = gravity_acceleration(r_values[n + 1])
        v_values[n + 1] = v_half + 0.5 * h * a_values[n + 1]

        # Calculate energy at this step
        energy_n = total_energy(r_values[n + 1], v_values[n + 1])
        energy_errors[n + 1] = abs(energy_n - initial_energy)  # Absolute error in energy

    return t_values, r_values, v_values, energy_errors
```

Leapfrog Method

$$\mathbf{x}_{i,n+1} = \mathbf{x}_{i,n} + \mathbf{v}_{i,n+1/2}\Delta t$$

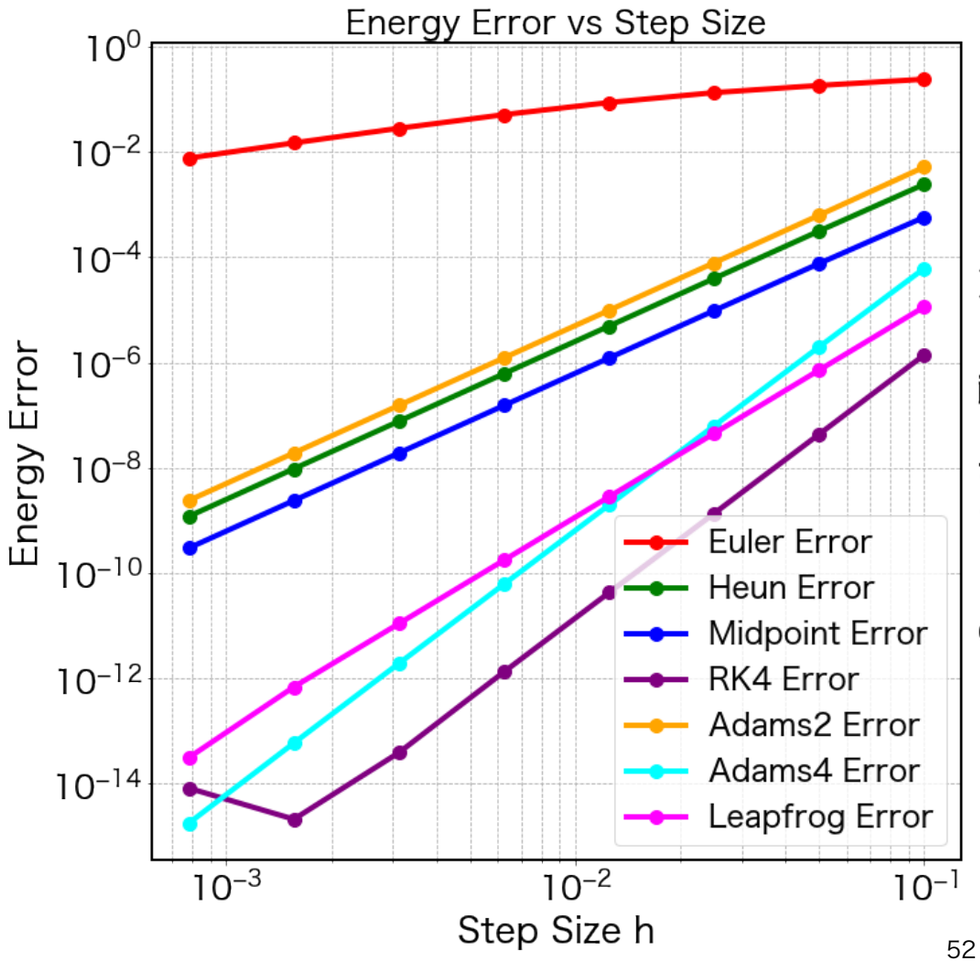
$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n+1/2} + \mathbf{a}_{i,n+1}\frac{\Delta t}{2}$$

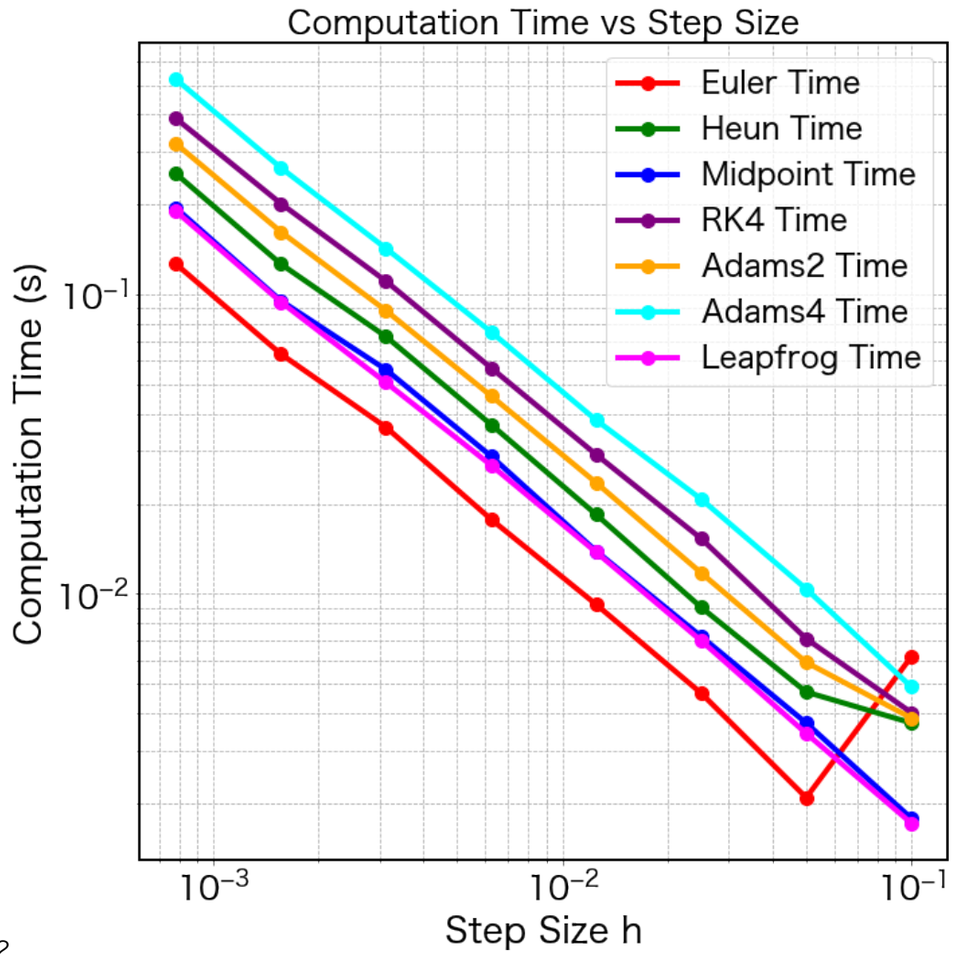
$$\mathbf{v}_{i,n+1/2} = \mathbf{v}_{i,n} + \mathbf{a}_{i,n}\frac{\Delta t}{2}$$

Ex.05) Kepler Problem (Two-body Problem)

$$\frac{d\mathbf{v}}{dt} = -GM\frac{\mathbf{x}}{|\mathbf{x}|^3}$$

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}$$



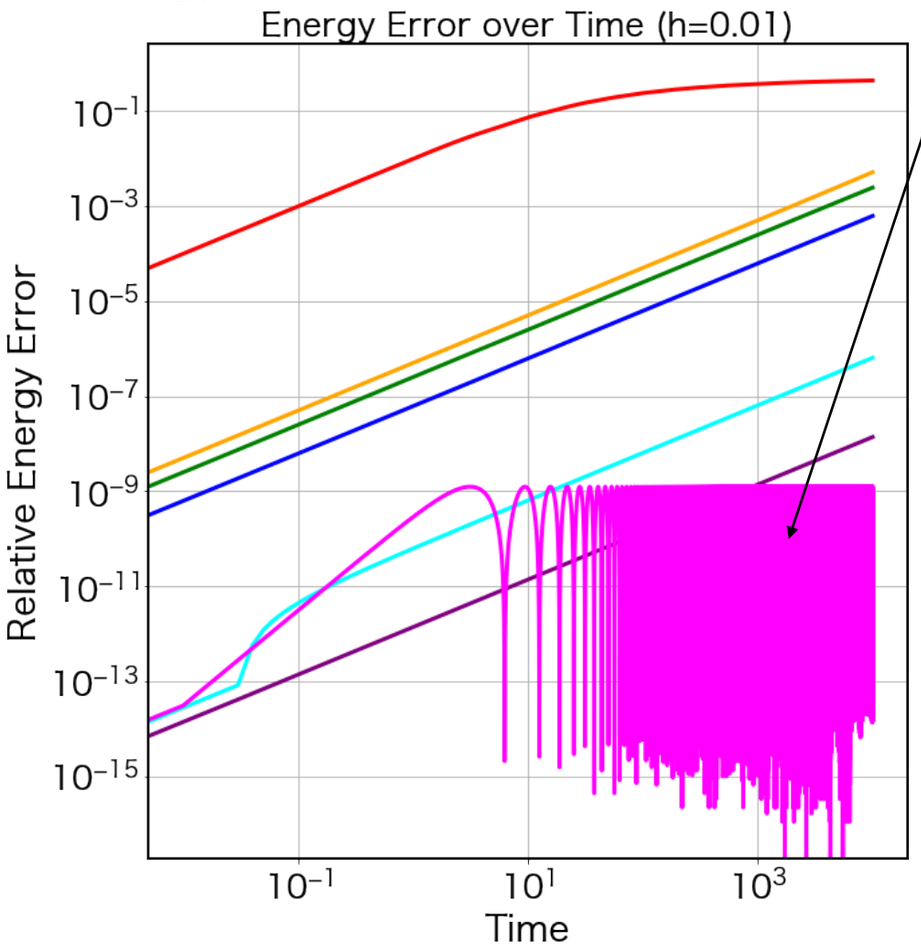


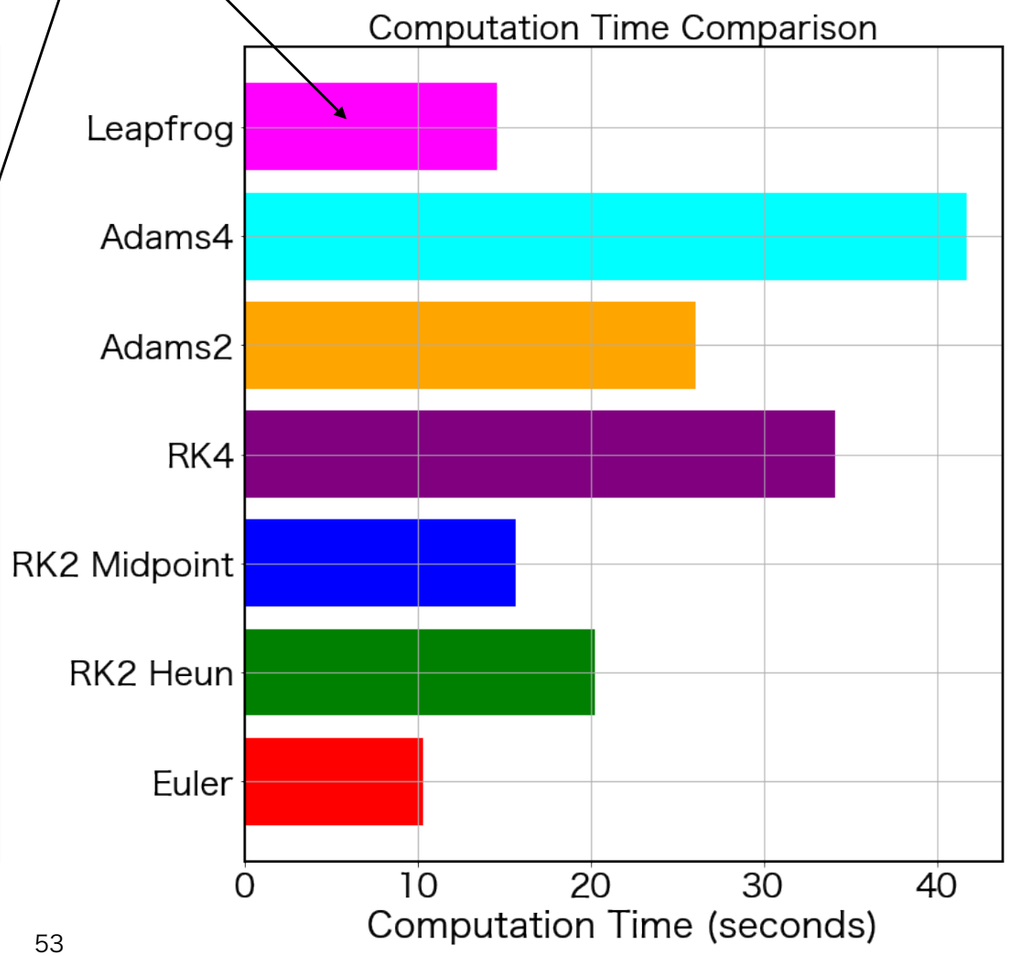
Ex.05) Kepler Problem (Two-body Problem)

$$\frac{d\mathbf{v}}{dt} = -GM\frac{\mathbf{x}}{|\mathbf{x}|^3}$$
$$\frac{d\mathbf{x}}{dt} = \mathbf{v}$$

Leapfrog method

fast, well E conservation





Hermite Integrators: Implicit Formula

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n h + \frac{1}{2}\dot{\mathbf{a}}_n h^2 + \frac{1}{6}\mathbf{a}_n^{(2)} h^3 + \frac{1}{24}\mathbf{a}_n^{(3)} h^4$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n h + \frac{1}{2}\mathbf{a}_n h^2 + \frac{1}{6}\dot{\mathbf{a}}_n h^3 + \frac{1}{24}\mathbf{a}_n^{(2)} h^4 + \frac{1}{120}\mathbf{a}_n^{(3)} h^5$$

where

$$\mathbf{a}_n^{(2)} = \frac{-6(\mathbf{a}_n - \mathbf{a}_{n+1}) - 2(2\dot{\mathbf{a}}_n + \dot{\mathbf{a}}_{n+1})h}{h^2}$$

$$\mathbf{a}_n^{(3)} = \frac{12(\mathbf{a}_n - \mathbf{a}_{n+1}) + 6(\dot{\mathbf{a}}_n + \dot{\mathbf{a}}_{n+1})h}{h^3}$$

Require $\mathbf{a}_{n+1}$ and $\dot{\mathbf{a}}_{\mathbf{n+1}}$ to calculate $\mathbf{a}_n^{(2)}$ and $\mathbf{a}_n^{(3)}$, i.e. the Hermite integrator is an **implicit** scheme. In practice, this difficulty is sidestepped by the **predictor-corrector method**.

Hermite Integrators: Derivation

3rd-order Hermite polynomial $a^{\text{poly}}(t) = k_0 + k_1(t - t_n) + k_2(t - t_n)^2 + k_3(t - t_n)^3$

conditions:

$$\begin{cases} a^{\text{poly}}(t_n) = k_0 = a_n \\ \dot{a}^{\text{poly}}(t_n) = k_1 = \dot{a}_n \\ a^{\text{poly}}(t_{n+1}) = k_0 + k_1 h + k_2 h^2 + k_3 h^3 = a_{n+1} \\ \dot{a}^{\text{poly}}(t_{n+1}) = k_1 + 2k_2 h + 3k_3 h^2 = \dot{a}_{n+1} \end{cases} \quad ==> \quad \text{coefficients:} \quad \begin{cases} k_0 = a_n \\ k_1 = \dot{a}_n \\ k_2 = \dfrac{-3(a_n - a_{n+1}) - (2\dot{a}_n + \dot{a}_{n+1})h}{h^2} \\ k_3 = \dfrac{2(a_n - a_{n+1}) + (\dot{a}_n + \dot{a}_{n+1})h}{h^3} \end{cases}$$

Thus,

$$\begin{cases} a^{\text{poly}}(t) = a_n + a_n(t - t_n) + \dfrac{1}{2}a_n^{(2)}(t - t_n)^2 + \dfrac{1}{6}a_n^{(3)}(t - t_n)^3, \text{ where } a^{(2)} = 2k_2 \text{ and } a^{(3)} = 6k_3 \\ v^{\text{poly}}(t) = v_n + \displaystyle\int_{t_n}^{t_{n+1}} a^{\text{poly}}(t)dt = v_n + a_n(t - t_n) + \frac{1}{2}\dot{a}_n(t - t_n)^2 + \frac{1}{6}a_n^{(2)}(t - t_n)^3 + \frac{1}{24}a_n^{(3)}(t - t_n)^4 \\ x^{\text{poly}}(t) = x_n + \displaystyle\int_{t_n}^{t_{n+1}} v^{\text{poly}}(t)dt = x_n + v_n(t - t_n) + \frac{1}{2}a_n(t - t_n)^2 + \frac{1}{6}\dot{a}_n(t - t_n)^3 + \frac{1}{24}a_n^{(2)}(t - t_n)^4 + \frac{1}{120}a_n^{(3)}(t - t_n)^5 \end{cases}$$

Hermite Integrator: PEC Algorithm

1. Calculate (Predict) $\mathbf{x}_p$ and $\mathbf{v}_p$ from $\mathbf{x}_n$, $\mathbf{v}_n$, $\mathbf{a}_n$ and $\dot{\mathbf{a}}_n$

$$\mathbf{v}_p = \mathbf{v}_n + \mathbf{a}_n h + \dot{\mathbf{a}}_n \frac{h^2}{2}$$

$$\mathbf{x}_p = \mathbf{x}_n + \mathbf{v}_n h + \mathbf{a}_n \frac{h^2}{2} + \dot{\mathbf{a}}_n \frac{h^3}{6}$$

2. Evaluate $\mathbf{a}_{n+1}$ and $\dot{\mathbf{a}}_{n+1}$ from $\mathbf{x}_p$ and $\mathbf{v}_p$, and evaluate $\mathbf{a}_n^{(2)}$ and $\mathbf{a}_n^{(3)}$ from $\mathbf{a}_n$, $\dot{\mathbf{a}}_n$, $\mathbf{a}_{n+1}$ and $\dot{\mathbf{a}}_{n+1}$ (gravity acceleration, jerk calculation)

3. Calculate (Correct) $\mathbf{x}_{n+1}$ and $\mathbf{v}_{n+1}$ from $\mathbf{x}_p$, $\mathbf{v}_p$, $\mathbf{a}_n^{(2)}$ and $\mathbf{a}_n^{(3)}$

$$\mathbf{v}_{n+1} = \mathbf{v}_p + \frac{1}{6}\mathbf{a}_n^{(2)} h^3 + \frac{1}{24}\mathbf{a}_n^{(3)} h^4$$

$$\mathbf{x}_{n+1} = \mathbf{x}_p + \frac{1}{24}\mathbf{a}_n^{(2)} h^4 + \frac{1}{120}\mathbf{a}_n^{(3)} h^5$$

1 evaluation of the gravity per 1 step

4. Go back to Step 1

Classical RK4 vs. Hermite Methods

Hermite method has the same $O(h^4)$ accuracy as RK4 method,
but requires **only one derivative (gravity) evaluation**, making it **faster**.

RK4 Method (Classical RK Method)

1. Calculate k_1 from t_n and y_n
2. Calculate k_2 from $t_n + \frac{h}{2}$, y_n and k_1
3. Calculate k_3 from $t_n + \frac{h}{2}$, y_n and k_2
4. Calculate k_4 from $t_n + h$, y_n and k_3
5. Update y_n to y_{n+1}
6. Go back to Step 1

4 evaluations of the derivative f (gravity) per 1 step

Hermite Method

1. Calculate $\mathbf{x}_p$ and $\mathbf{v}_p$ from $\mathbf{x}_n$, $\mathbf{v}_n$, $\mathbf{a}_n$ and $\dot{\mathbf{a}}_n$
2. Evaluate $\mathbf{a}_{n+1}$ and $\dot{\mathbf{a}}_{n+1}$ from $\mathbf{x}_p$ and $\mathbf{v}_p$, and evaluate $\mathbf{a}_n^{(2)}$ and $\mathbf{a}_n^{(3)}$ from $\mathbf{a}_n$, $\dot{\mathbf{a}}_n$, $\mathbf{a}_{n+1}$ and $\dot{\mathbf{a}}_{n+1}$
3. Calculate $\mathbf{x}_{n+1}$ and $\mathbf{v}_{n+1}$ from $\mathbf{x}_p$, $\mathbf{v}_p$, $\mathbf{a}_n^{(2)}$ and $\mathbf{a}_n^{(3)}$
4. Go back to Step 1.

1 evaluation of the gravity per 1 step

Summary: Time Integration Methods

- **Gravitational *N*-body system**
  - Planetary system, star cluster, galaxy, cluster of galaxies, universe
  - The EoS is a 2nd-order ODE in 3N dim, but it can be solved numerically as a 1st-order ODE in 6N dim.

$$\frac{d^2\mathbf{x}_i}{dt^2} = \sum_{j=1,j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} \quad ==> \quad \begin{cases} \dfrac{d\mathbf{v}_i}{dt} = \displaystyle\sum_{j=1,j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} \\ \dfrac{d\mathbf{x}_i}{dt} = \mathbf{v}_i \end{cases}$$

3N dim. 2nd-order ODEs
(3N次元連立2階常微分方程式)

6N dim. simultaneous 1st-order ODEs
(6N次元連立1階常微分方程式)

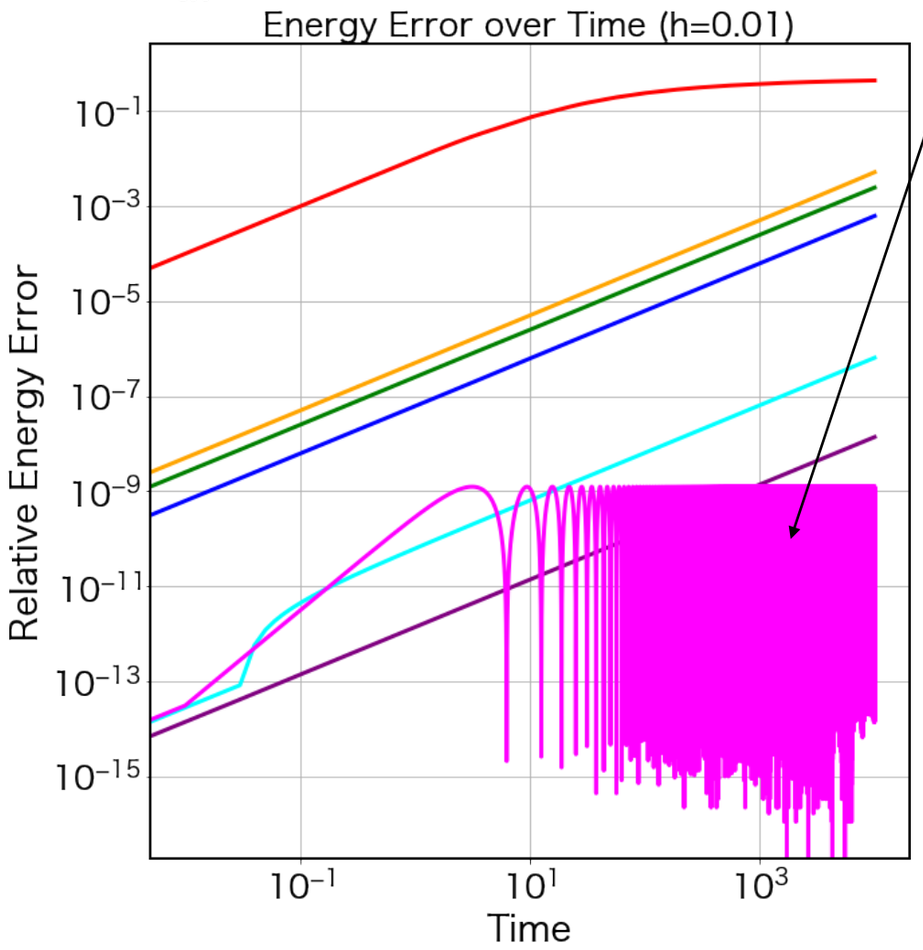
- **Collisional** / **Collisionless** Systems
  - Two-body relaxation time: $t_{\rm relax} \equiv \dfrac{v^2}{\langle \Delta v_\perp^2 \rangle} \sim \dfrac{v^3}{G^2 n m^2 \ln\Lambda} \sim 0.1 \dfrac{N}{\ln N} t_{\rm cross}$
  - Collisionless systems: e.g. galaxy
  - Collisional systems: e.g. star cluster, planetary system
- Time integration methods
  - **Leap-frog** Method (for collisionless): $O(\Delta t^2)$, 1 evaluation of gravity per step, symplectic
  - **Hermite** Method (for collisional): $O(\Delta t^4)$, 1 evaluation of gravity per step

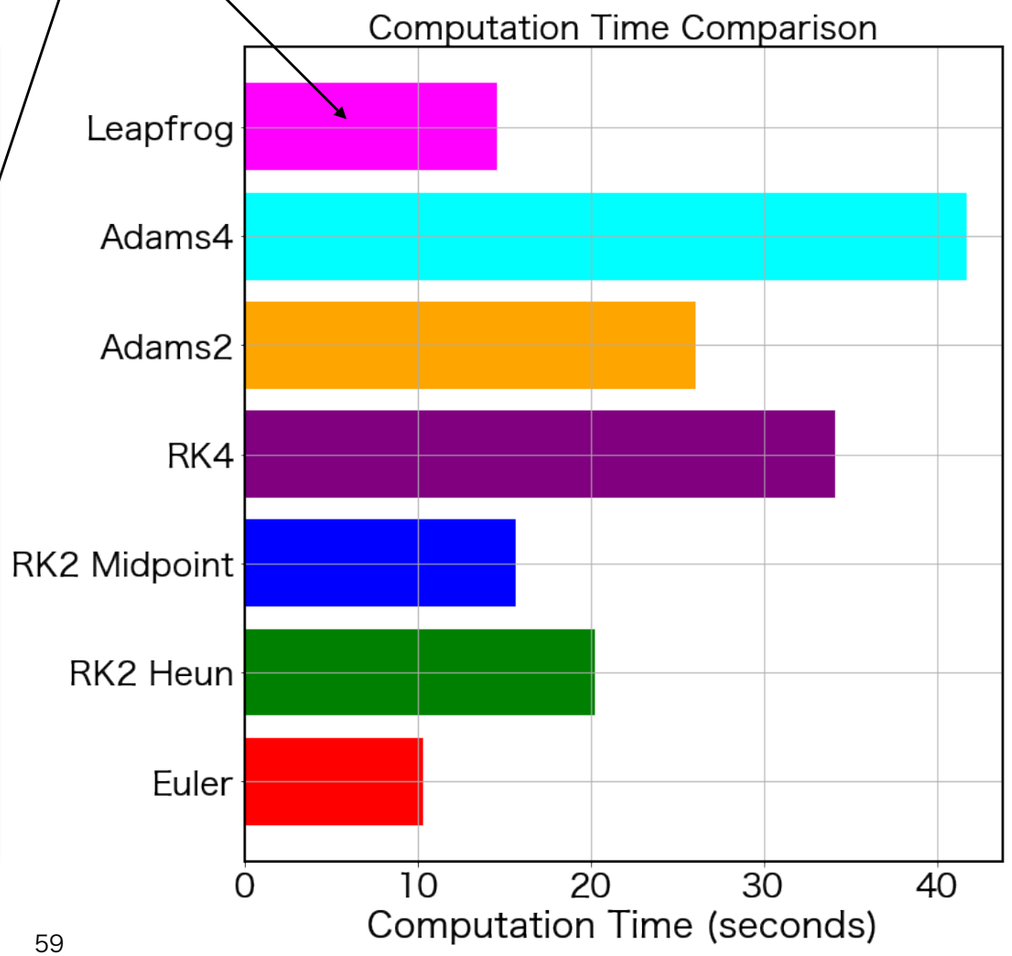
Ex.05) Kepler Problem (Two-body Problem)

$$\frac{d\mathbf{v}}{dt} = -GM\frac{\mathbf{x}}{|\mathbf{x}|^3}$$
$$\frac{d\mathbf{x}}{dt} = \mathbf{v}$$

Leapfrog method:

Fast and good E conservation





Lecture #3 (2024/11/8)

Numerical Methods for N-body Systems 2: Gravitational Interactions

(重力相互作用計算法)

1. Gravity Calculation: Direct and Tree Methods
2. Gravity Calculation: Grid-based Methods
3. Gravity Calculation: Basis Functional Expansion
4. Advanced: Sub-grid Models for Galaxy Simulations

N-Body Simulations

Last Week: 2-body problem (Kepler problem)

This Week: Many-body problem (*N*>>10)

Equations of Motion:

$$\frac{d^2\mathbf{x}_i}{dt^2} = \sum_{j=1,j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}$$

==>

Converting to 1st-Order ODEs

$$\begin{cases} \dfrac{d\mathbf{v}_i}{dt} = \boxed{\displaystyle\sum_{j=1,j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}} \\ \dfrac{d\mathbf{x}_i}{dt} = \mathbf{v}_i \end{cases}$$

6N dimensional simultaneous 1st-order ODEs

(6N次元連立1階常微分方程式)

Difficulties

- LHS: Accurate time integration to treat **wide dynamical timescale**
- RHS: Large calc. cost, $O(N^2)$, due to **gravitational interactions**

Gravity Force Calculation

Collisional System (衝突系):

- **Direct Summation** (直接和)

  e.g. planetary system, planetesimal system, star cluster

Collisionless System (無衝突系):

- **Tree Method** (ツリー法) = Approximate Direct Summation

  e.g. galaxy
- Particle-Mesh (PM) Method w/ FFT (粒子-メッシュ法)

  e.g. cosmological simulation
- Spherical Harmonic Expansion Method (球面調和関数展開法)

  e.g. spherical galaxy, dark matter halo
- **Basis Function Expansion Method** (基底関数展開法)

  e.g. spherical galaxy, dark matter halo

Since these algorithms are complex,
it's highly recommended to use existing open-source code rather than developing your own.

Learning Goals

Background:

- Advanced simulation codes for simulation astronomy are already available (such as open-source codes).
  (シミュレーション天文学の研究に利用する本格的なコードはすでに存在する)
- Instead of developing code from scratch, the focus should be on using existing codes effectively to achieve research outcomes.
  (コードを1から自分で書くというよりは、既存の公開コードを適切に使って、研究成果を出すことが重要)

Learning Goals:

- Learn to view code not as a black box, but to understand its internal processes.
  (基本：コードをブラックボックスとしてではなく、内部処理を類推できるようになる)
- [Advanced] Develop skills to tune and optimize codes.
  (発展：プログラムのチューニングができるようになる)

Direct Summation

Gravity acceleration in an *N*-body system:

$$\mathbf{a}_i = \sum_{j=1, j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}$$ (exact form in Newtonian gravity)

Calculation Accuracy and Cost:

- Accuracy: **Perfect or very high accuracy**
- Cost: $O(N^2)$

Fast Calculation:

- **GRAPE (GRAvity PipE)**: Special-purpose computer for gravitational many-body problem (重力多体問題専用計算機). Hardware
- **GPU (Graphic Processing Unit)**: Handling numerous threads simultaneously due to thousands of cores, massive parallelism, high memory bandwidth hardware
- **GRAPE Emulator (Phantom-GRAPE)**: a “software”-based alternative for environments without access to GRAPE hardware

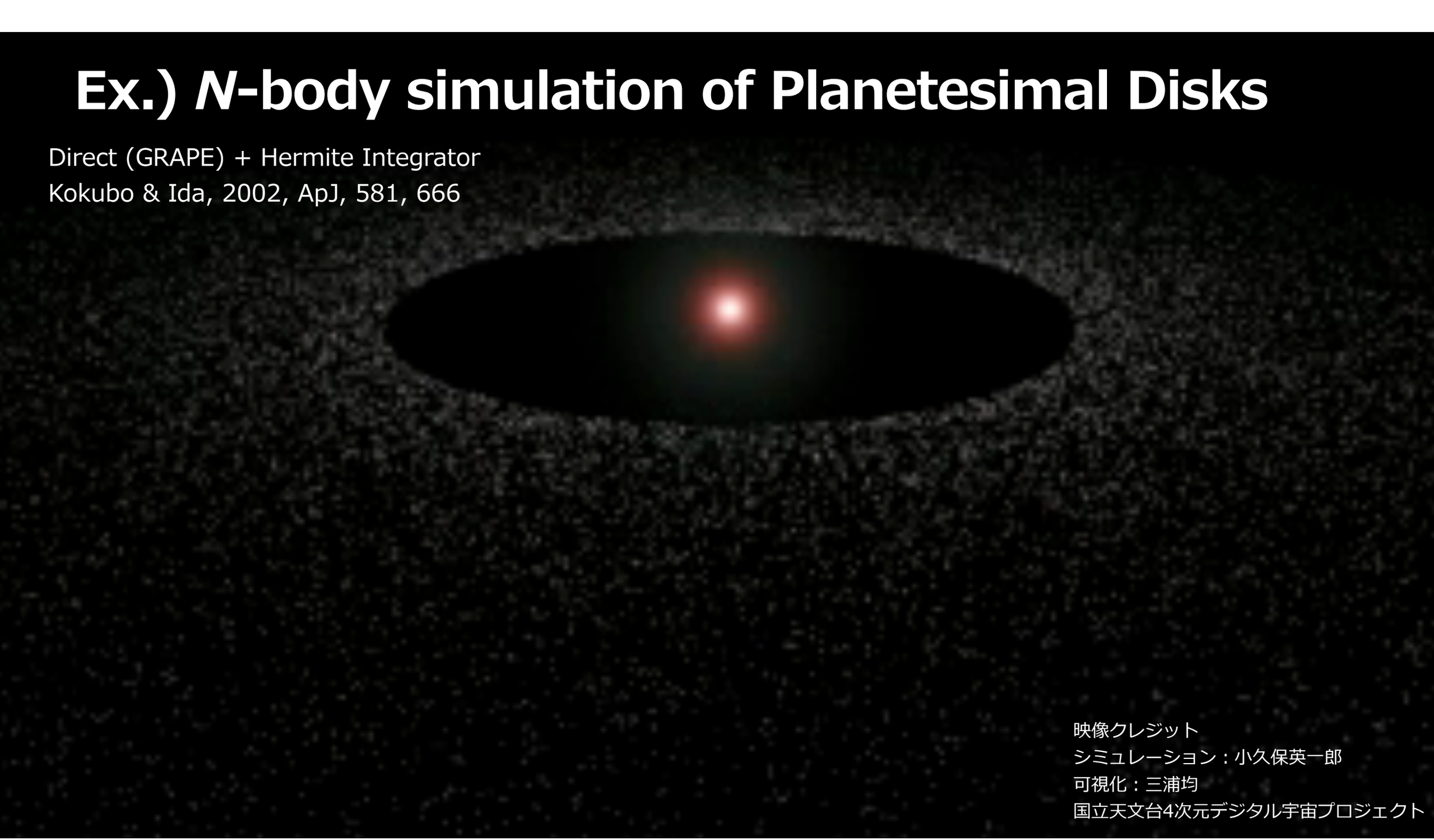
Direct Summation: Very Easy Algorithm!

A double loop is necessary to calculate directly gravitational interactions for each pair of particles. This double-loop approach has a computational complexity of $O(N^2)$, meaning that as the number of particles grows, the computation becomes significantly more demanding.

```python
def compute_gravitational_forces(positions, masses):
    """Calculate gravitational interactions for all particles and return the total force on each particle."""
    forces = np.zeros_like(positions)

    for i in range(N):
        for j in range(i + 1, N):  # Calculate each pair only once with j > i
            # Distance vector and its magnitude
            r_vec = positions[j] - positions[i]
            distance = np.linalg.norm(r_vec)

            if distance == 0:
                continue  # Skip self-interaction

            # Calculate force using Newton's law of gravitation
            force_magnitude = G * masses[i] * masses[j] / distance**2
            force_vector = force_magnitude * r_vec / distance  # Force vector with direction

            # Add force to each particle
            forces[i] += force_vector
            forces[j] -= force_vector  # Reaction force

    return forces
```

Ex.) *N*-body simulation of Planetesimal Disks

Direct (GRAPE) + Hermite Integrator
Kokubo & Ida, 2002, ApJ, 581, 666



映像クレジット
シミュレーション：小久保英一郎
可視化：三浦均
国立天文台4次元デジタル宇宙プロジェクト

Tree Method: Approximate Direct Summation

From **all** particles

(すべての「粒子」からの重力を計算)

$$\mathbf{a}_i = \sum_{j=1, j\neq i}^{N} Gm_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}$$

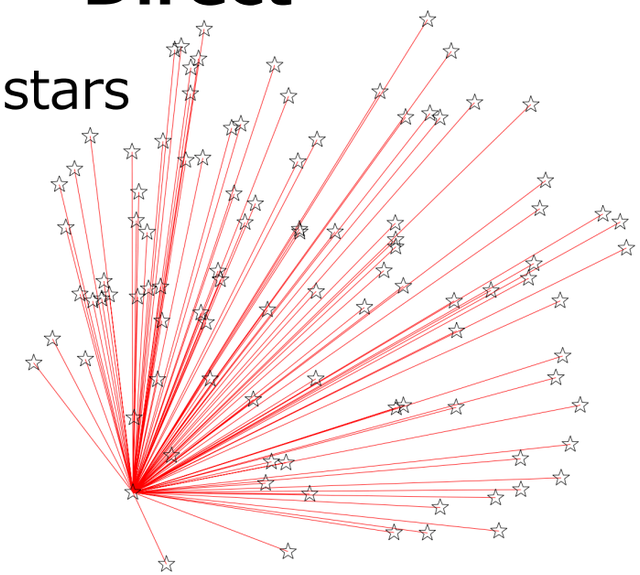
Binding of distant particles

(遠方粒子をひとまとめにして計算量を減らす)

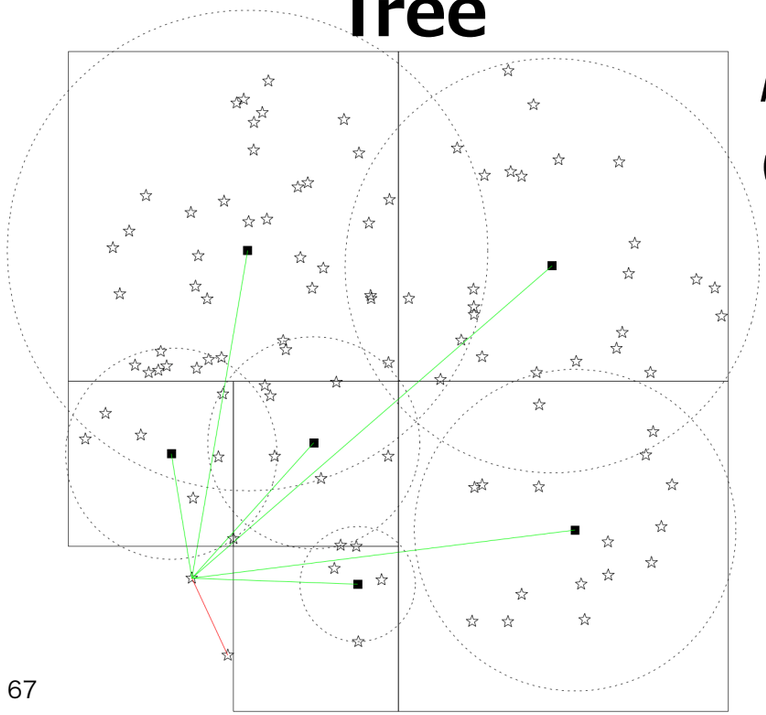
$$\mathbf{a}_i = \sum_{k=1}^{N_{\mathrm{cell}}} Gm_j \frac{\mathbf{x}_k - \mathbf{x}_i}{|\mathbf{x}_k - \mathbf{x}_i|^3}$$

Direct

(N-1) stars

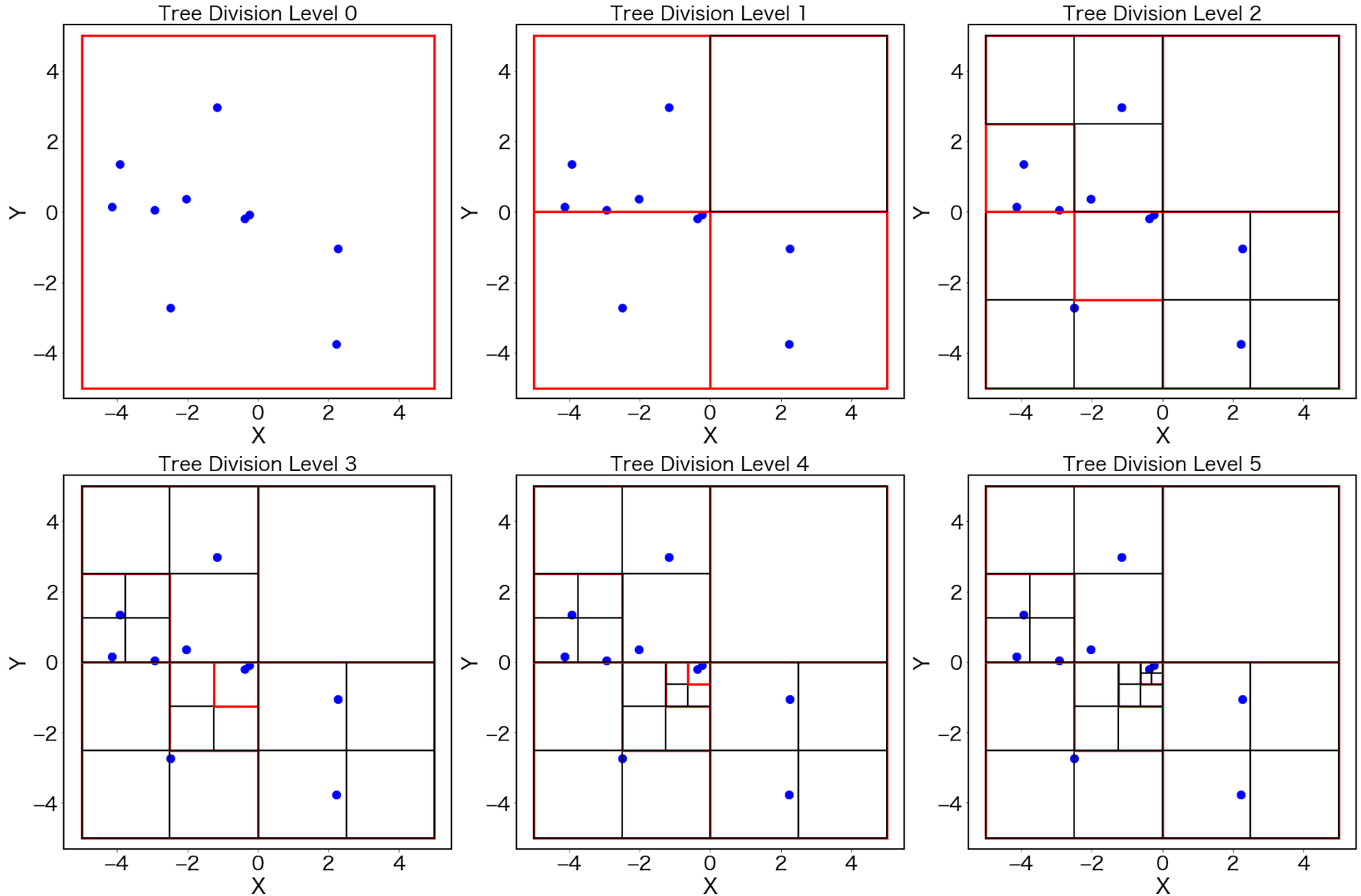


Tree

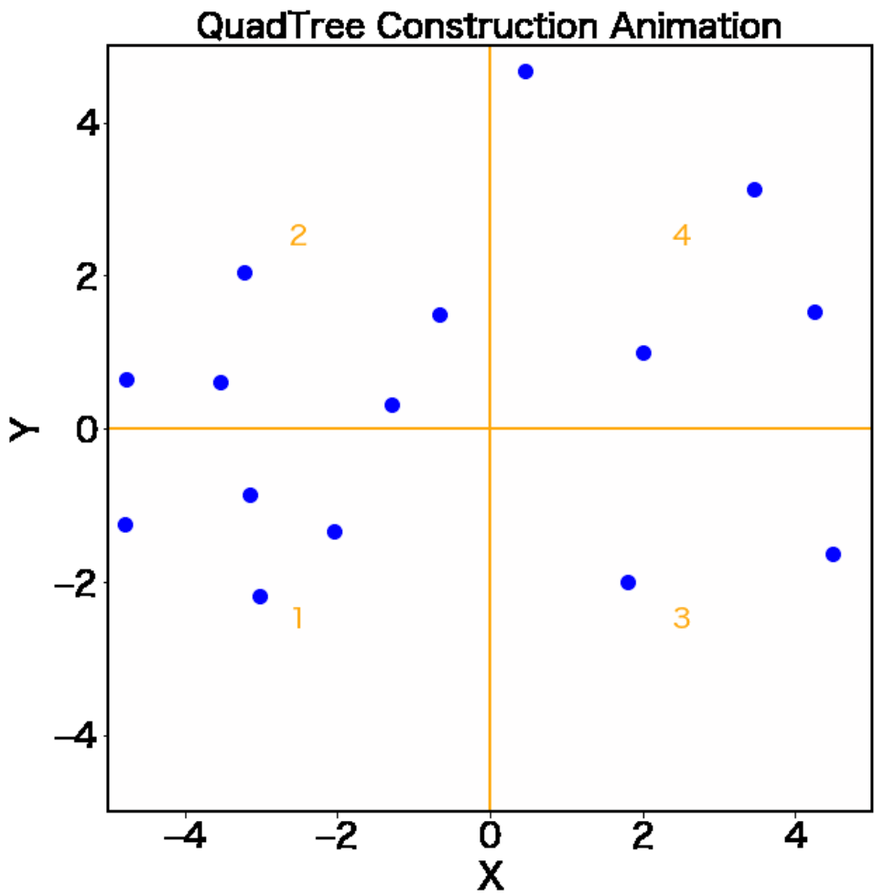


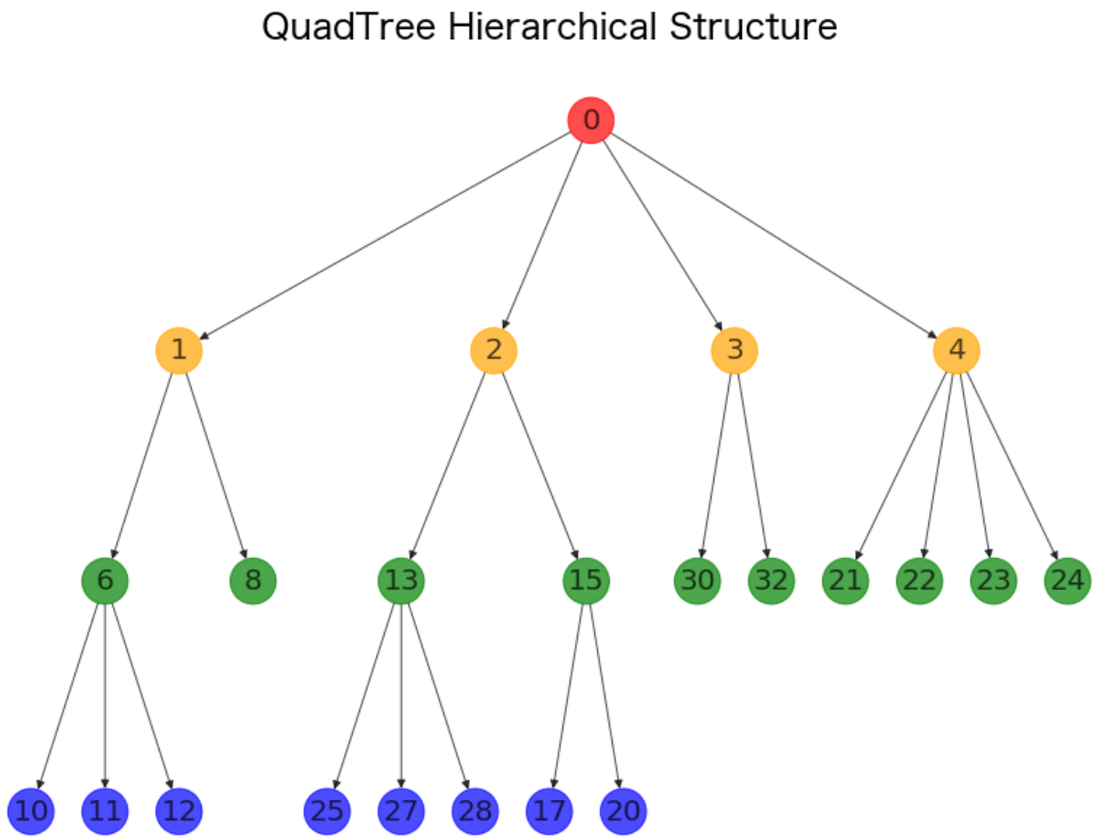
N_{cell} Cells

(~log$N$)


Tree Division Level 0
Tree Division Level 1
Tree Division Level 2
Tree Division Level 3
Tree Division Level 4
Tree Division Level 5
X
Y
−4
−2
0
2
4

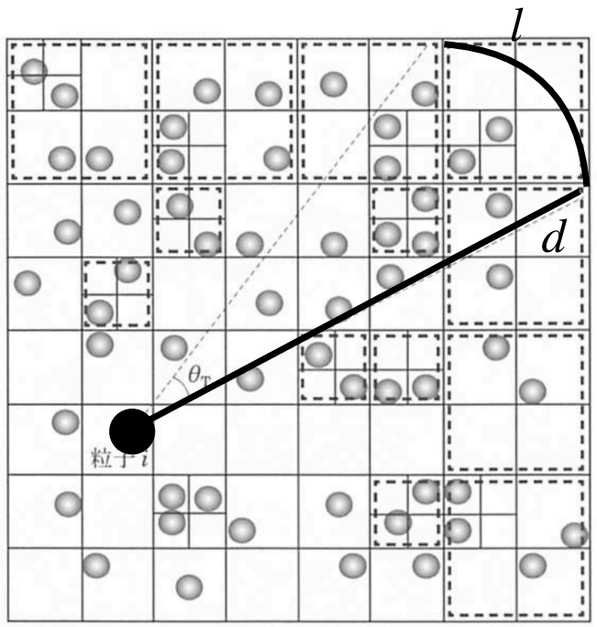
Tree Method: Barns & Hut Algorithm



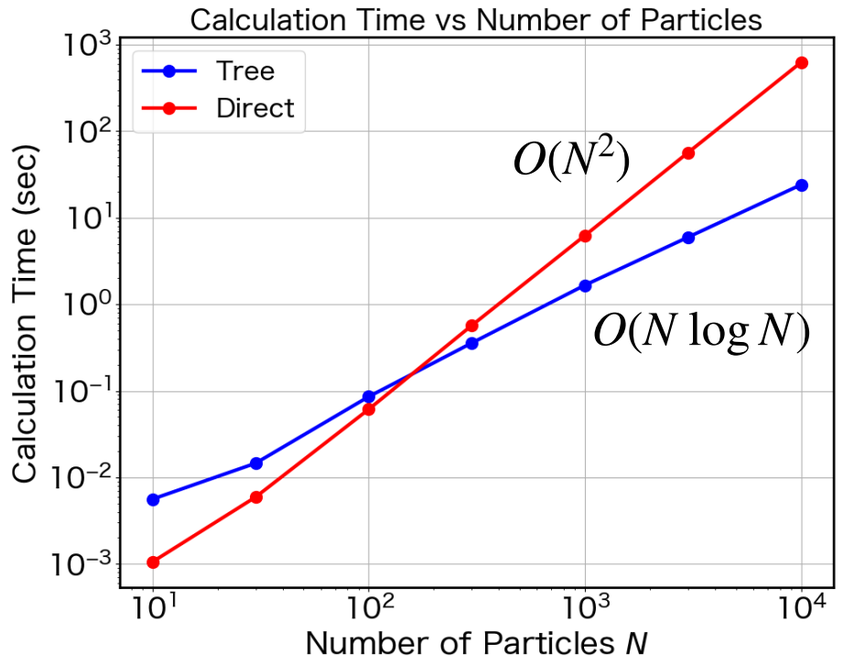


Tree Method: Barns & Hut Algorithm

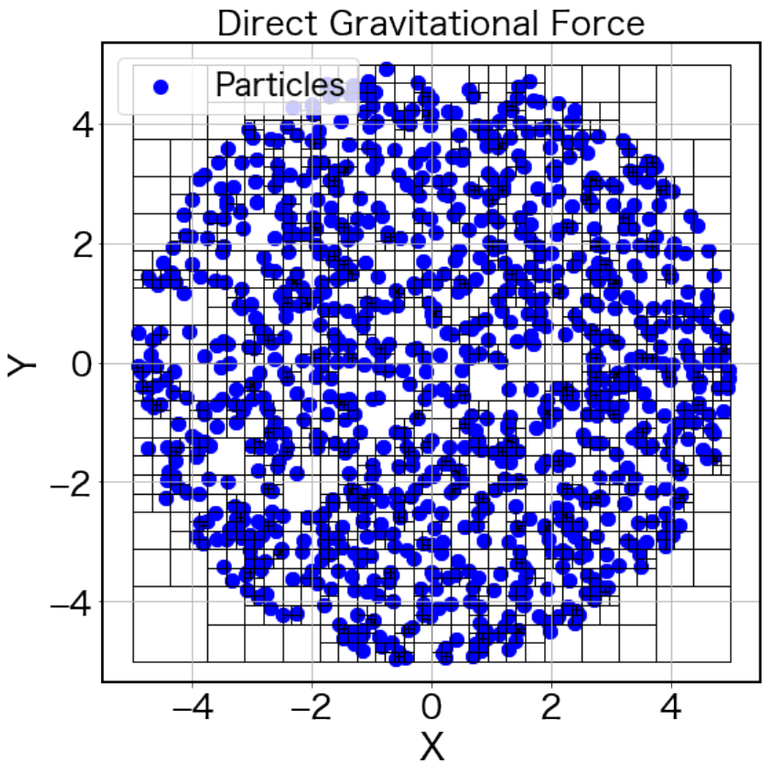
1. Construct “tree” structure by cell division.
2. Calculate mass and center-of-mass (COM) inside a cell.
3. Calculate gravity force on a particle from a cell.
   - if $\frac{l}{d} < \theta_T$: force from a cell (d: distance to cell COM; l: cell length)
   - otherwise: force from child cells

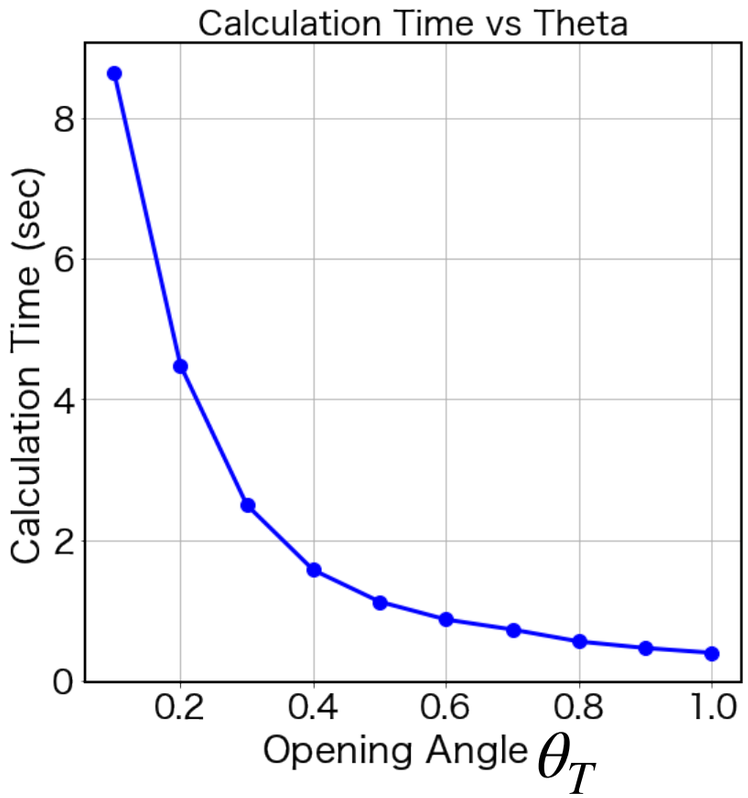


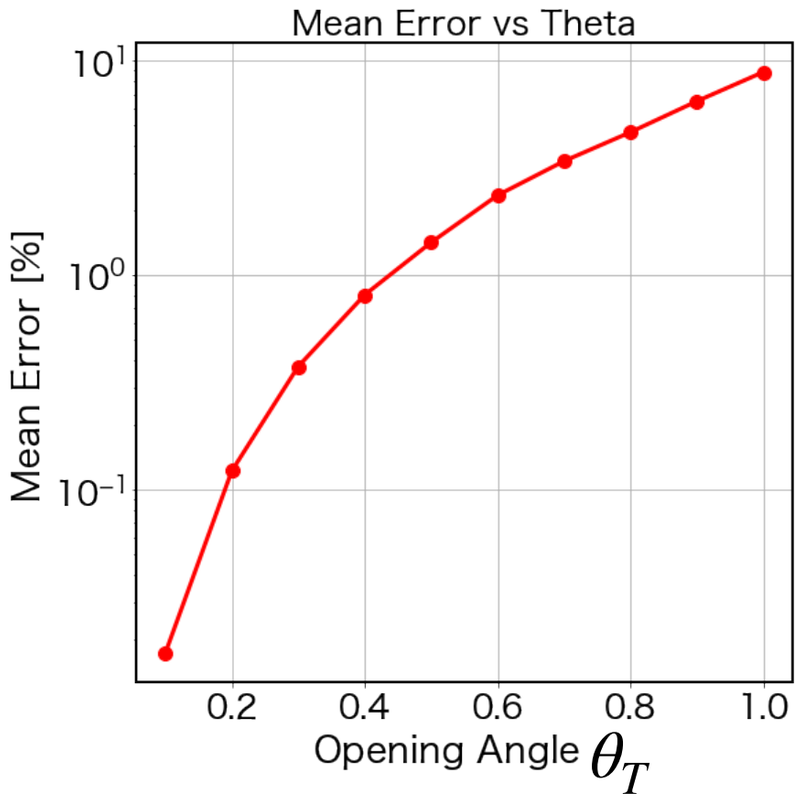
$$\theta_{cell} = \frac{l}{d}$$



Ex.06) Barns-Hut Tree Method







usually $\theta_T \sim 0.1 - 0.3$

YouTube JP　検索　天の川銀河紀行

NAOJ/CfCA
Center for Computational Astrophysics

N-body/SPH code "**ASURA**" (Saitoh et al. 2008; Saitoh & Makino 2009)
Tree method + Leapfrog Integrator

可視化：中山弘敬・国立天文台4D2U
Data : Baba, MNRAS, 454, 2954, (2015); Baba et al. MNRAS, 464, 246, (2017)
Code : ASURA (Saitoh et al. 2008; Saitoh & Makino 2009, 2010)
https://4d2u.nao.ac.jp/t/var/download/MWJourney.html

4D2U
国立天文台4次元デジタル宇宙プロジェクト
FOUR-DIMENSIONAL DIGITAL UNIVERSE PROJECT, NAOJ

YouTube JP 検索 天の川銀河変動史

N-body/SPH code “**ASURA**” (Saitoh et al. 2008; Saitoh & Makino 2009)
Tree method + Leapfrog Integrator

「アテルイ II」CfCA/NAOJ

可視化：中山弘敬 (4D2U)

Baba, Kawata & Schoenrich (2022)
Baba & Kawata (2020)

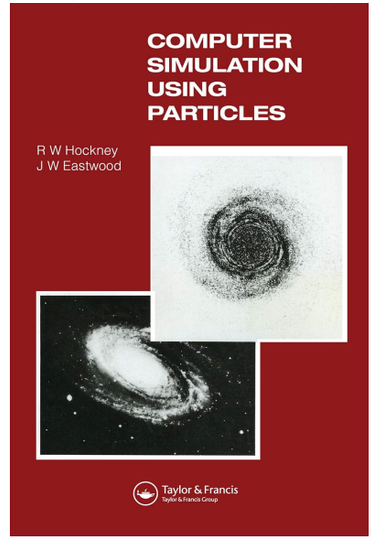
©2022 Junichi Baba, Hirotaka Nakayama, 4D2U Project, NAOJ

4D2U 国立天文台4次元デジタル宇宙プロジェクト FOUR-DIMENSIONAL DIGITAL UNIVERSE PROJECT, NAOJ

Grid-based Method: Particle-Mesh Method

Classical Text

Hockney & Eastwood (1988)



Solve Poisson equation w/ FFT

$$\nabla^2\Phi(\mathbf{x}) = 4\pi G\rho(\mathbf{x})$$

$$= \textbf{Fourier Transformation} \Rightarrow \quad \widetilde{\Phi}(\mathbf{k}) = -\frac{4\pi G}{k^2}\widetilde{\rho}(\mathbf{k})$$

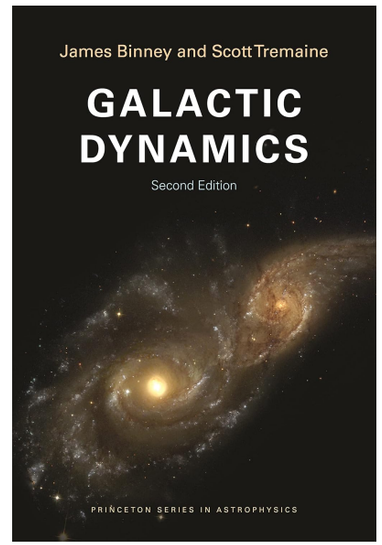
Cost: $\sim O(N \log N)$

Applications:

- **Periodic boundary** (e.g. cosmology)
- Vacuum boundary, but increases computational complexity and cost (i.e. requiring more memory)
- Particle-Particle PM Method (**P³M method**)

  Combines PM for long-range forces with direct particle-particle (PP) calculations for short-range interactions.

Binney & Tremaine (2008)



Particle-Mesh Method: Algorithm

1. **Assigning Mass to the Mesh** (e.g. Cloud-in-Cell, **CIC method**):

   i) particle mass $m_i \rightarrow$ density field $\rho^{\mathrm{grid}}(\mathbf{x})$

   ii) FFT: $\rho^{\mathrm{grid}}(\mathbf{x}) \rightarrow \widetilde{\rho}(\mathbf{k})$

2. Calculating **Gravitational Potential**:

   i) $\widetilde{\Phi}(\mathbf{k}) = -\dfrac{4\pi G}{k^2}\widetilde{\rho}(\mathbf{k})$

   ii) Inverse FFT: $\widetilde{\Phi}(\mathbf{k}) \rightarrow \Phi^{\mathrm{grid}}(\mathbf{x})$

3. Calculating **Gravitational Acceleration**:

   i) grav. acc. in mesh w/ numerical diff. e.g. $a_x^{\mathrm{grid}} = -\dfrac{\Phi^{\mathrm{grid}}(x+\Delta x, y, z) - \Phi^{\mathrm{grid}}(x-\Delta x, y, z)}{2\Delta x}$

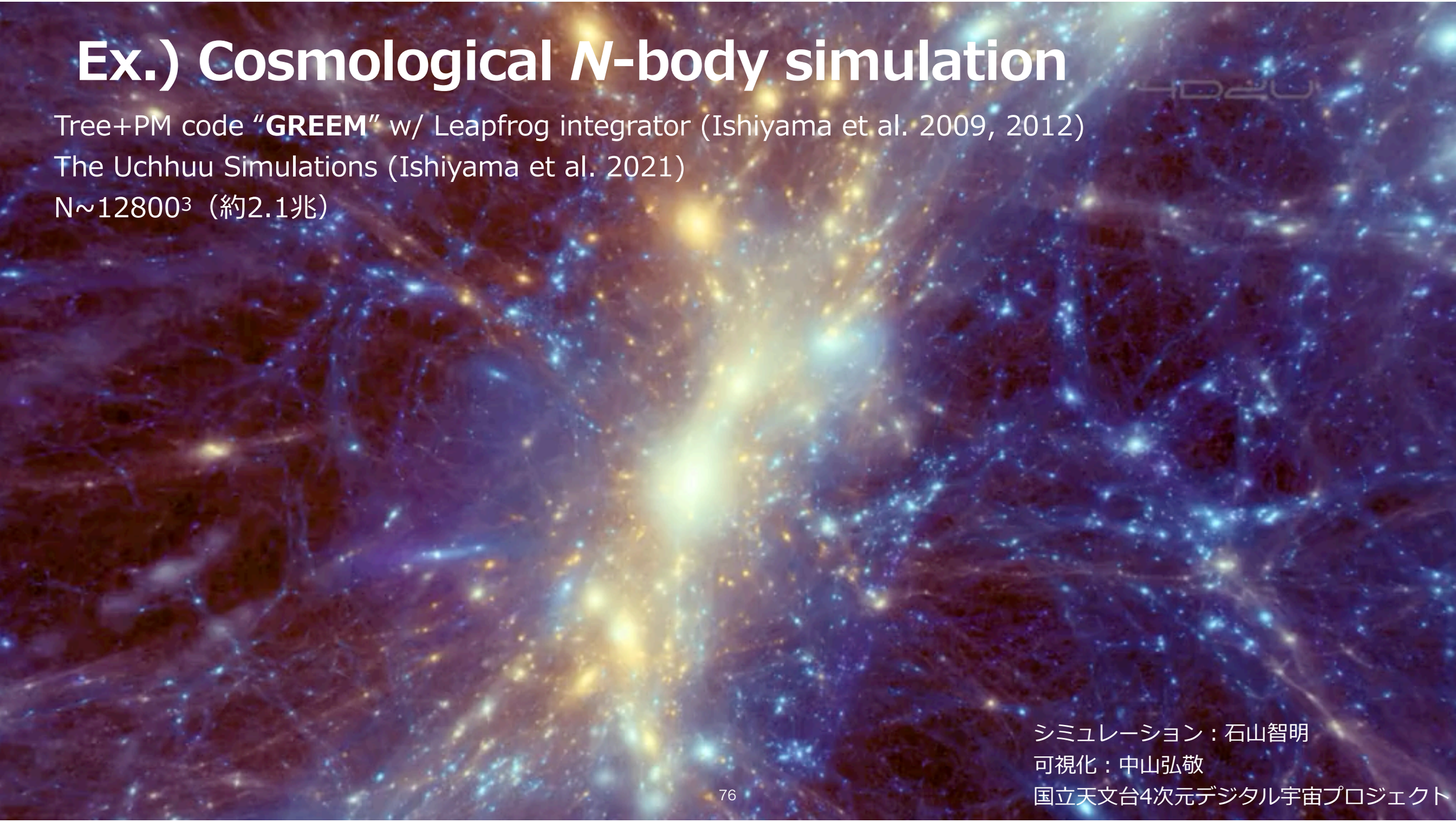
   ii) Interpolating grid accelerations to *i*-th particle, $\mathbf{a}(\mathbf{x}_i)$

Ex.) Cosmological *N*-body simulation

Tree+PM code “**GREEM**” w/ Leapfrog integrator (Ishiyama et al. 2009, 2012)

The Uchhuu Simulations (Ishiyama et al. 2021)

N~12800^3（約2.1兆）



シミュレーション：石山智明

可視化：中山弘敬

国立天文台4次元デジタル宇宙プロジェクト

Spherical Harmonic Expansion (球面調和関数展開)

Multi-pole expansion (多重極展開)

Solve Poisson equation w/ SHE

$$\nabla^2\Phi(\mathbf{x}) = 4\pi G\rho(\mathbf{x})$$

$\rho(\mathbf{x})$ and $\Phi(\mathbf{x})$ expand with spherical harmonics:

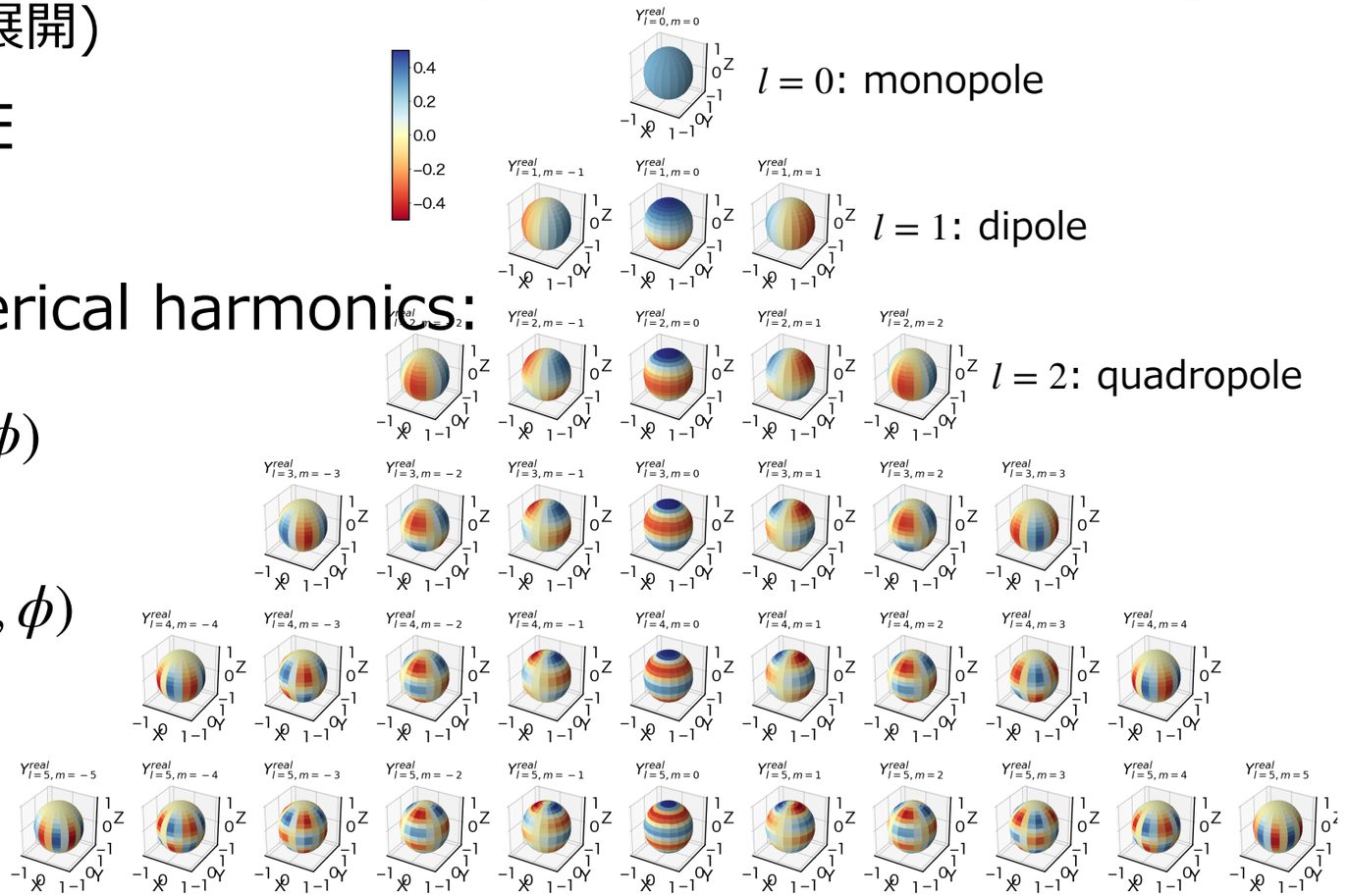
$$\rho(r,\theta,\phi) = \sum_{l=0}^{\infty}\sum_{m=-l}^{m=l}\rho_{l,m}(r)Y_{l,m}(\theta,\phi)$$

$$\Phi(r,\theta,\phi) = \sum_{l=0}^{\infty}\sum_{m=-l}^{m=l}\Phi_{l,m}(r)Y_{l,m}(\theta,\phi)$$

Then,

$$\Phi_{l,m}(r) = -\frac{4\pi G}{2l+1}Q_{l,m}(r)$$

$$Q_{l,m}(r) = \frac{1}{r^{l+1}}\int_0^r \rho_{l,m}(a)a^{l+2}da + r^l\int_r^{\infty}\frac{\rho_{l,m}(a)}{a^{l-1}}da$$



$l = 0$: monopole

$l = 1$: dipole

$l = 2$: quadropole

← numerical integration of $\rho_{l,m}(r)$

$l = 0$: monopole, $l = 1$: dipole,

$l = 2$: quadropole, ⋯

Spherical Harmonic Expansion: Algorithm

1. **Assigning Mass to the Mesh** (e.g. Cloud-in-Cell, **CIC method**):

   i) particle mass $m_i \rightarrow$ density field $\rho^{\text{grid}}(\mathbf{x})$

   ii) calc. density coefficients: $\rho_{l,m}(r) = \int_0^{2\pi} \int_0^{\pi} \rho(r,\theta,\phi) Y^*_{l,m}(\theta,\phi) \sin\theta d\theta d\phi$ (using numerical integrations)

2. Calculating **Gravitational Potential**:

   i) numerical integration: $Q_{l,m}(r) = \frac{1}{r^{l+1}} \int_0^r \rho_{l,m}(a) a^{l+2} da + r^l \int_r^{\infty} \frac{\rho_{l,m}(a)}{a^{l-1}} da$

   ii) calc. potential coefficients: $\Phi_{l,m}(r) = -\frac{4\pi G}{2l+1} Q_{l,m}(r)$

   iii) calc. potential: $\Phi^{\text{grid}}(r,\theta,\phi) = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^{m=l} \Phi_{l,m}(r) Y_{l,m}(\theta,\phi)$

3. Calculating **Gravitational Acceleration**:

   i) calc. accelerations $a_r^{\text{grid}}(r,\theta,\phi)$, $a_\phi^{\text{grid}}(r,\theta,\phi)$, $a_\theta^{\text{grid}}(r,\theta,\phi)$ in mesh from $\Phi^{\text{grid}}(r,\theta,\phi)$

   ii) Interpolating grid accelerations to *i*-th particle, $\mathbf{a}(\mathbf{x}_i)$

Basis Function Expansion Method (基底関数展開法)

e.g. General Fourier Expansion (一般フーリエ展開)

$\rho(\mathbf{x})$ and $\Phi(\mathbf{x})$ expand with basis functions:

$$\rho(r,\theta,\phi) = \sum_{n=0}^{\infty}\sum_{l=0}^{\infty}\sum_{m=-l}^{m=l} \rho_{n,l}(r) Y_{l,m}(\theta,\phi)$$

$$\Phi(r,\theta,\phi) = \sum_{n=0}^{\infty}\sum_{l=0}^{\infty}\sum_{m=-l}^{m=l} \Phi_{n,l}(r) Y_{l,m}(\theta,\phi)$$

The basis functions $\{\rho_{n,l}\}$ and $\{\Phi_{n,l}\}$ do not need to be orthogonal, respectively.

Substituting these expansion forms into the Poisson eq. $\nabla^2\Phi(\mathbf{x}) = 4\pi G\rho(\mathbf{x})$:

$$\frac{1}{r^2}\frac{\mathrm{d}}{\mathrm{d}r}\left(r^2\frac{\mathrm{d}\Phi_{n,l}}{\mathrm{d}r}\right) - \frac{l(l+1)}{r^2}\Phi_{n,l} - 4\pi G\rho_{n,l} = 0.$$

Thus, it reduces to the problem of finding the basis function sets $\{\rho_{n,l}\}$ and $\{\Phi_{n,l}\}$ that satisfy this ODE.

Basis Function Expansion Method (基底関数展開法)

Condsider s_n defined as:

$$s_n \equiv \int \Phi^*_{n,l}(r) Y^*_{l,m}(\theta,\phi)\rho(\mathbf{x})\mathrm{d}^3\mathbf{x} = \int \Phi^*_{n,l}(r) Y^*_{l,m}(\theta,\phi) \sum_{n',l',m'} \rho_{n',l'}(r) Y_{l',m'}(\theta,\phi)\mathrm{d}^3\mathbf{x}$$

$$= \sum_{n',l',m'} \int \Phi^*_{n,l}(r)\rho_{n',l'}(r)\mathrm{d}r \int Y^*_{l,m}(\theta,\phi) Y_{l',m'}(\theta,\phi)\sin\theta \mathrm{d}\theta\mathrm{d}\phi = \sum_{n',l',m'} \int \mathrm{d}r \Phi^*_{n,l}(r)\rho_{n',l'}(r)\delta_{l,l'}\delta_{m,m'}$$

$$= \sum_{n'} \int \Phi^*_{n,l}(r)\rho_{n',l}(r) r^2 \mathrm{d}r = \sum_{n'} M_{n,n'}$$

Here, the matrix $M_{n',n} \equiv \int \Phi^*_{n,l}(r)\rho_{n',l}(r) r^2 \mathrm{d}r$ is an integral over r, so it does not depend on the density distribution $\rho(\mathbf{x})$.

If $\{\rho_{n,l}\}$ and $\{\Phi_{n,l}\}$ are **orthogonal** (i.e. **biorthogonal basis set; 双直交基底関数**), the matrix $M_{n',n} = \int \Phi^*_{n,l}(r)\rho_{n',l}(r) r^2 \mathrm{d}r \propto \delta_{n',n}$ becomes diagonal. In other words, the radial modes n and n' are independent.

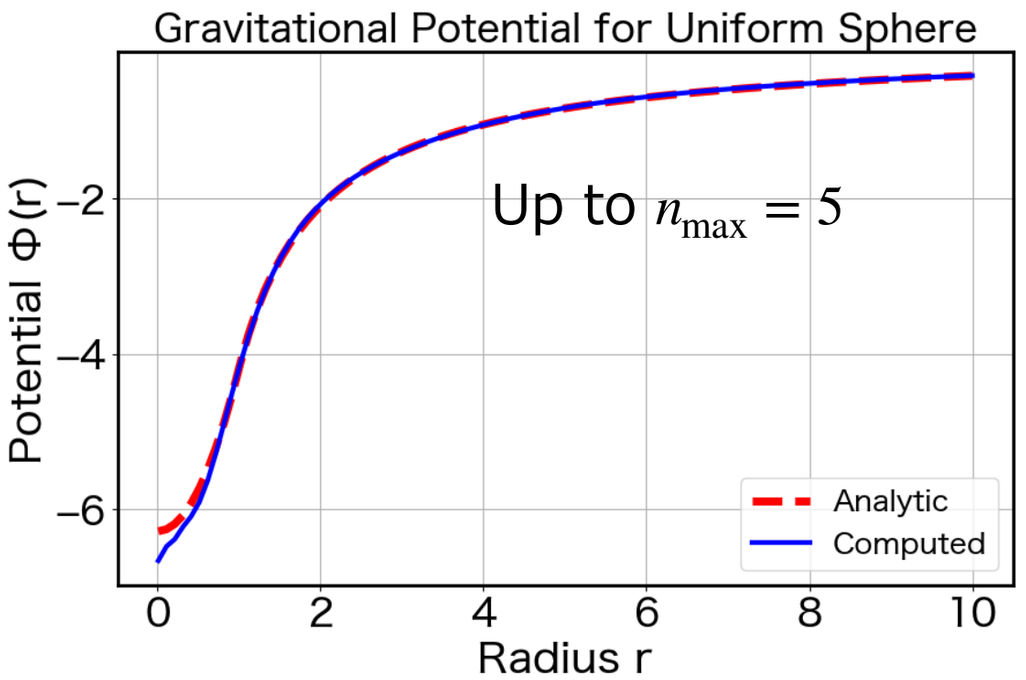
Ex.08) Basis Function Sets

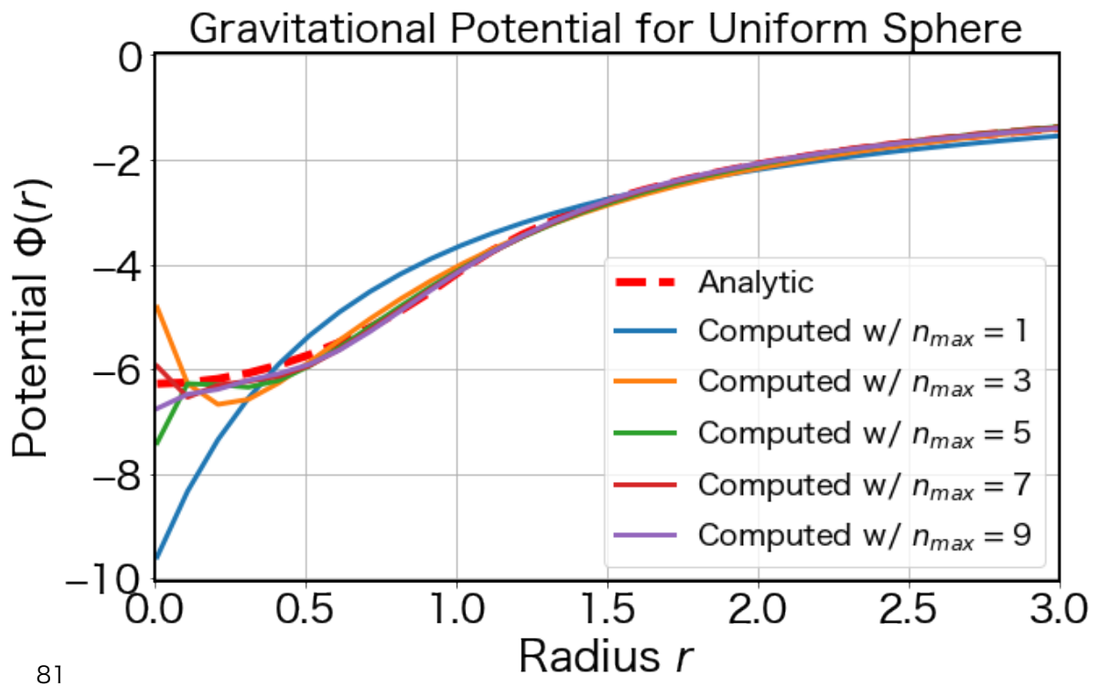
Hernquist & Ostriker Basis Set (Hernquist & Ostriker 1992):

$$\rho_{n,l}(r) = \frac{K_{n,l}}{2\pi} \frac{r^l}{r(1+r)^{2l+3}} C_n^{(2l+3/2)}\left(\frac{r-1}{r+1}\right)$$

$$\Phi_{n,l}(r) = -\frac{r^l}{(1+r)^{2l+1}} C_n^{(2l+3/2)}\left(\frac{r-1}{r+1}\right)$$

where $C_n^{\alpha}(x)$ is the Gegenbauer polynomial.

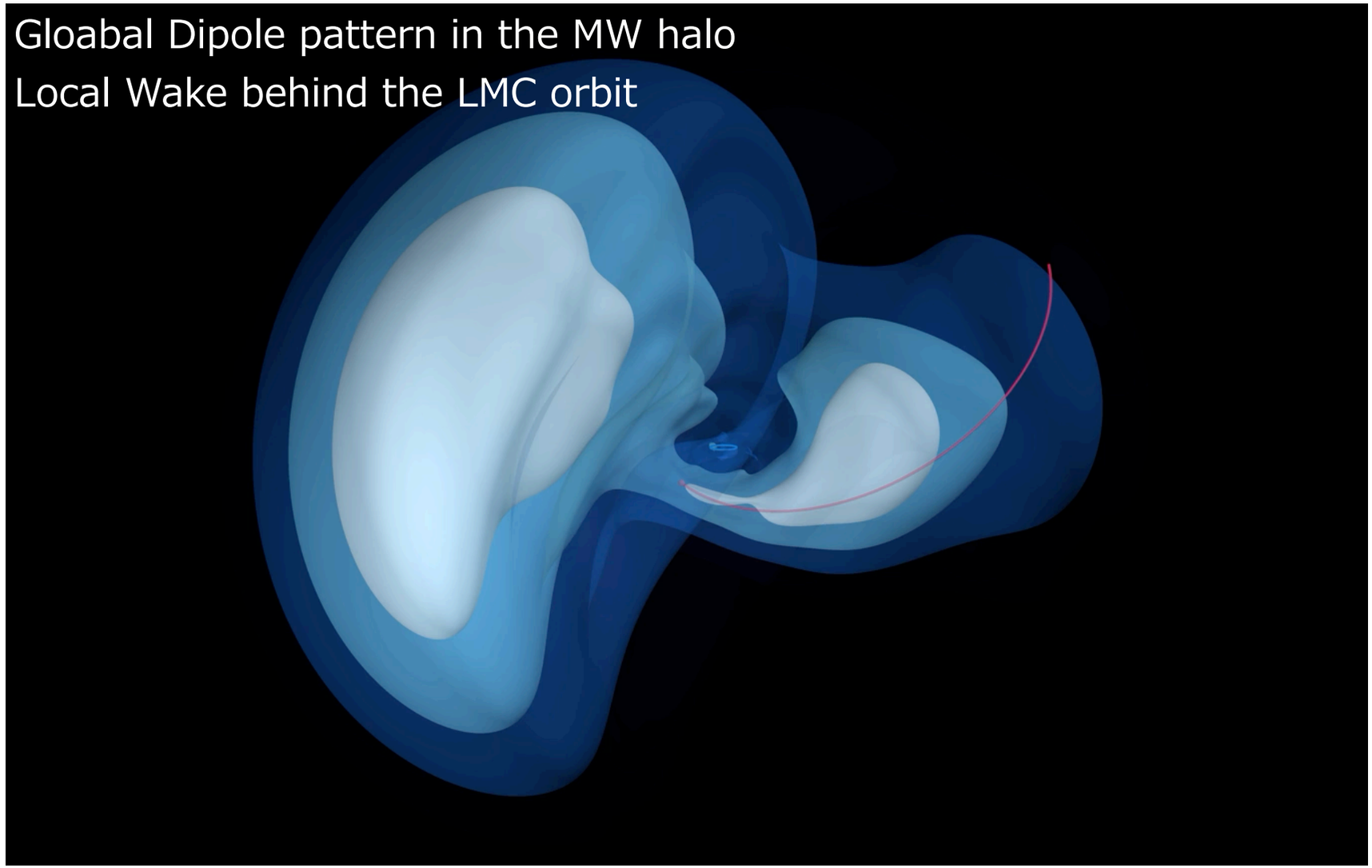




Ex) BFE Analysis of MW-LMC Interactions

using Hernquist & Ostriker BFE

Garavito-Camargo et al. (2021)



Summary: Gravity Calculation Methods

- **Collisional System (衝突系):**
  - **Direct Summation**:
    - accuracy: perfect
    - cost: $O(N^2)$
- **Collisionless System (無衝突系):**
  - **Tree Method** (ツリー法): approximate direct summation
    - accuracy: depending on opening angle θ_T (usually $\theta_T \sim 0.1 - 0.3$)
    - cost: $O(N \log N)$
    - e.g. galaxy simulations
  - **Particle-Mesh (PM) Method** w/ FFT (粒子-メッシュ法)
    - accuracy: depending on mesh size
    - e.g. cosmological simulations
  - **Spherical Harmonic Expansion/Basis Function Expansion**
    - accuracy: depending on number of expansion terms (n, l, m)
    - e.g. post-process analysis of *N*-body sim.

Since these algorithms are complex,
it's highly recommended to use ***existing open-source codes***
rather than developing your own.

Open-Source Codes

- **ASURA**: *N*-body/SPH code for galaxy formation (by T. Saitoh, 神戸大学 斎藤貴之; C)
  - Available only for “collaborative” research.
  - If you’re interested, please contact me, Michiko Fujii (U. of Tokyo), or Takayuki Saitoh (Kobe U.).
- **FDPS**: a general-purpose, high-performance library for particle simulations (Fortran/C)
  - https://github.com/FDPS/FDPS
  - There is a workshop once a year (contact J. Makino from “tennet” for details).
- **NBODY6**: *N*-body code for collisional systems (by S. Aarseth; Fortran)
  - https://people.ast.cam.ac.uk/~sverre/web/pages/nbody.htm
- **PeTar**: *N*-body code for collisional systems (by L. Wang; C++)
  - https://github.com/lwang-astro/PeTar
- **galpy** : Test particle simulation code (by J. Bovy; Python)
  - https://github.com/jobovy/galpy
  - Useful for orbital calculations with Gaia data.
- **AGAMA** : Test particle simulations and various gravitational potential calculations (E. Vasiliev; C++)
  - https://github.com/GalacticDynamics-Oxford/Agama
  - Very useful for generating equilibrium initial conditions for galaxy simulations and analyzing gravitational fields in N-body simulation results.

Advanced: Collisionless+Collisional “Hybrid” Method

Bridge code (Bridge is for Realistic Interactions in Dense Galactic Environment; M. Fujii et al. 2006)

- Field Stellar Particles:Leapfrog + Tree
- Cluster Stellar Particles: Hermite + Direct

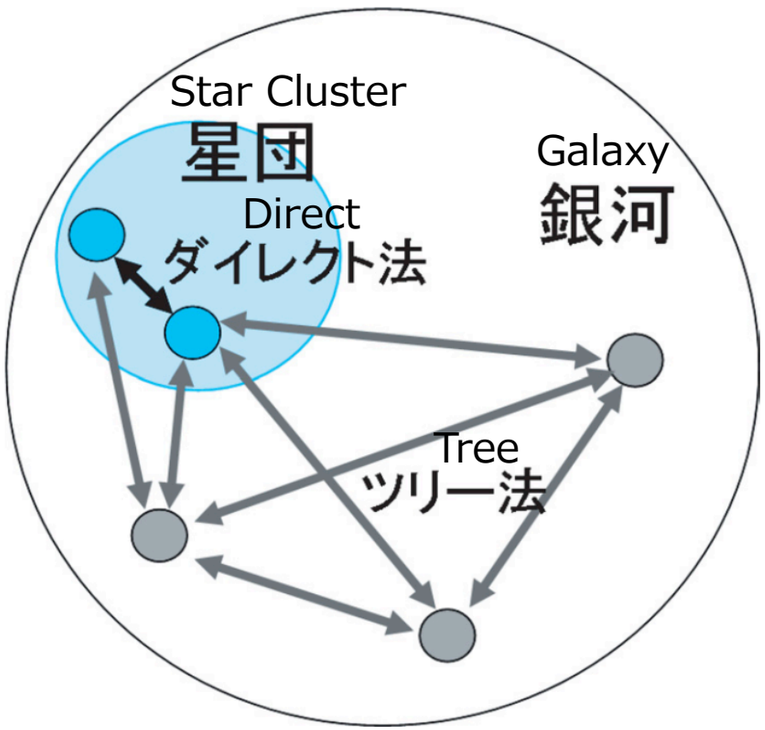
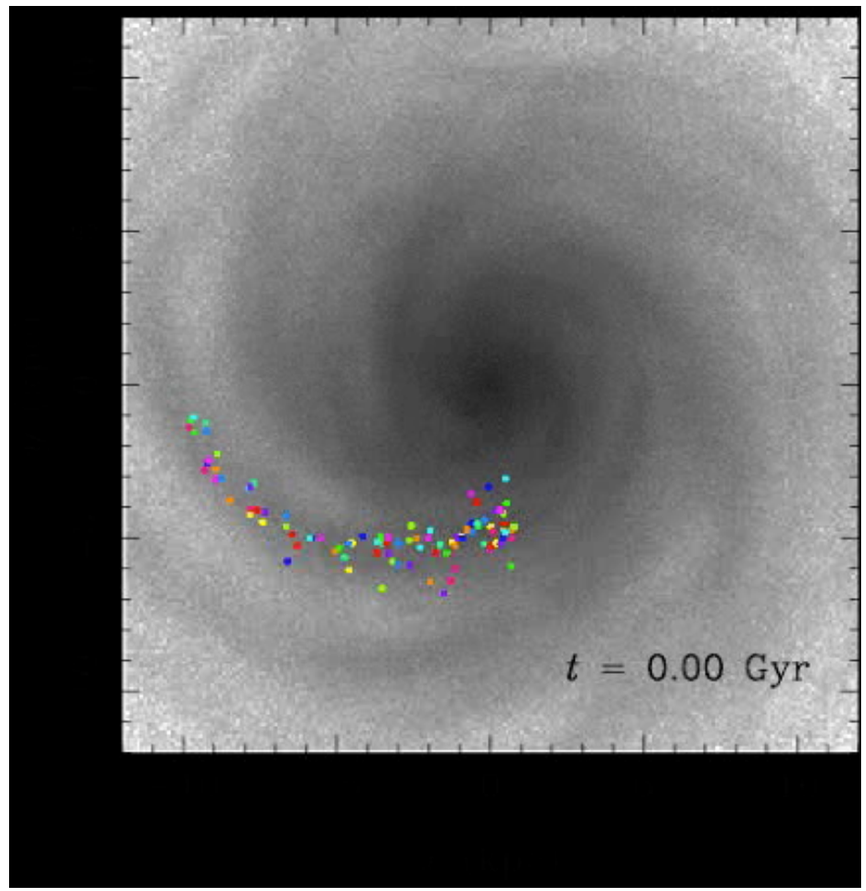


図3　ハイブリッド法の概念図.

藤井通子、天文月報2007年9月号

Destruction of star clusters due to radial migration in dynamic spiral galaxies



Fujii & Baba (2012), *MNRAS Letters*, 427, L16

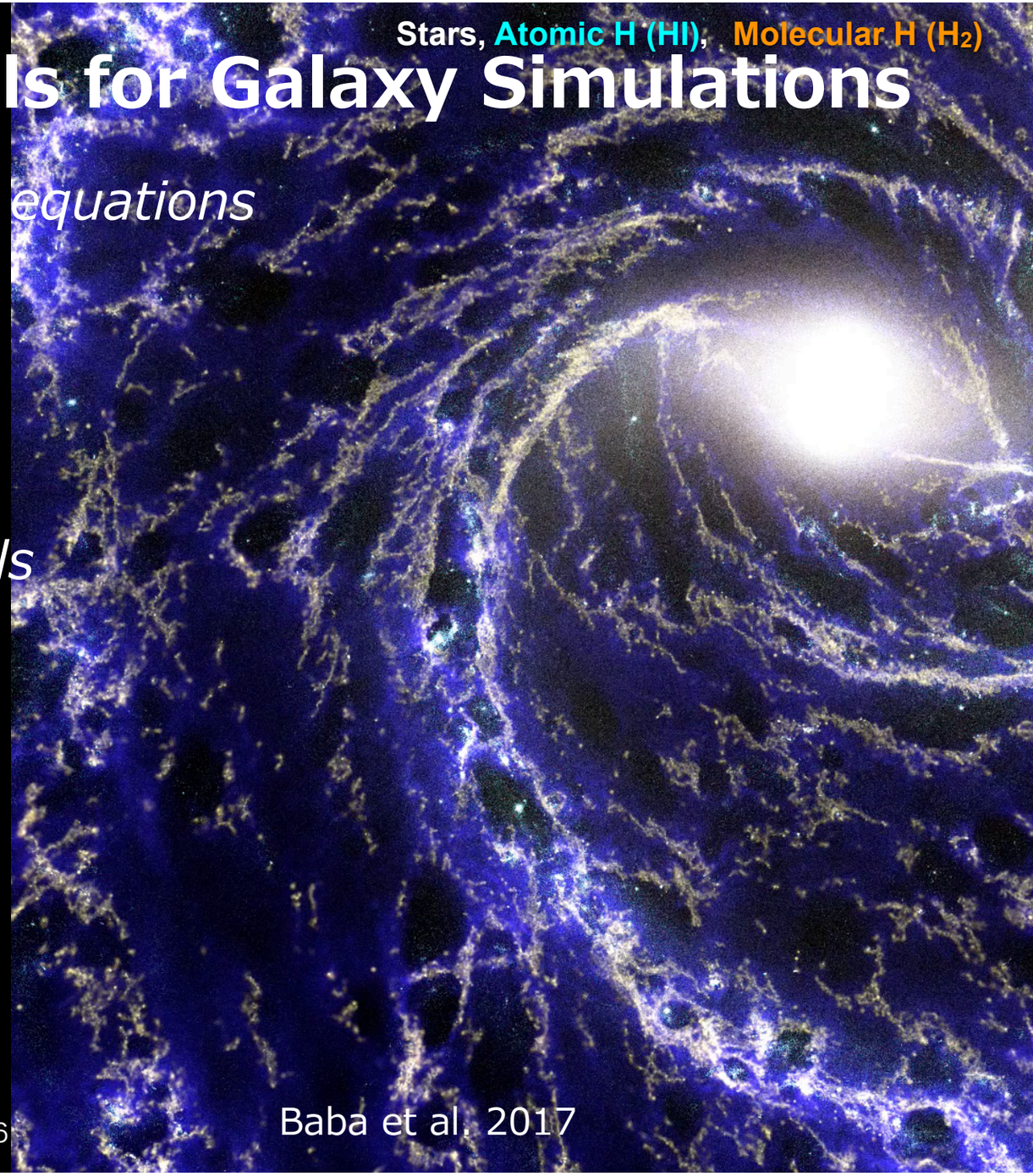
Advanced: Sub-grid Models for Galaxy Simulations

Dynamics (Physics) : *Solving basic equations*

- ✓**Collisionless dynamics** (stars, DM)
- ✓**Collisional dynamics** (star cluster)
- ✓**Fluid dynamics** (gas)
- ✓**Thermal dynamics of ISM**
  Radiative cooling/heating ($10<T<10^8$ K)

"Sub-grid" models : *ad-hoc models*

- ✓**Star Formation**
  - Molecular clouds (~10 K)
- ✓**Stellar Feedback** (stars → gas)
  - HII regions
  - Supernova explosions (SNe II, SNe Ia)
  - Stellar wind from AGB stars
  - Chemical enrichment
- ✓**AGN Feedback**
  - Radiation
  - Jet / Disk wind



Baba et al. 2017

Advanced: Sub-grid models

- **3D Galactic Structure:**
  - Stellar disk: scale-radius ~ a few kpc, scale-height ~ 100 pc
  - Gaseous disk: scale-radius ~ a few kpc, scale-height ~ 10 pc
  - Molecular Clouds/Star Clusters: <~ 10 pc, <~ 10^4 $M_\odot$
- **Typical resolutions for Galaxy (formation/evolution) simulations**
  - ~$10^{6\text{-}7}$ stellar particles/gal. -> ~$10^{3\text{-}4}$ $M_\odot$/particle (MW-size gal.)
  - gravitational softening ~ 10-100 pc

Resolutions have not yet reached a level sufficient to fully resolve these structures.
==> **Star formation / feedback cannot be solved from first principles**
==> Requiring **subgrid models**

Advanced: Sub-grid models

- **Star formation** (e.g. Katz 1992; Saitoh et al. 2008)

  if $\rho_{\rm gas} > 100\ {\rm H/cm^3}$, $T_{\rm gas} < 100\ {\rm K}$, $\nabla \cdot \mathbf{v} < 0$,

  Schmidt law: $\frac{d\rho_{\rm SF}}{dt} = c_\star \rho_{\rm gas}^{1/2}$

  where the SF efficiency parameter $c_\star \approx 0.03$.

  Stellar particles are generated probabilistically from gas to satisfy the Schmidt law.

- **Simple Stellar Population (SSP):** star particle = star cluster

  (1) Identical Age, (2) Uniform Chemical Composition,

  (3) Initial Mass Function (e.g. Salpeter-type IMF with a stellar mass range of [0.1, 100]$M_\odot$

- **Stellar Feedback**

  - SN energy feedback: smoothly distributed over the surrounding gas (Satioh et al. 2008)
  - Heavy elements released from stellar evolution are similarly added to the surrounding gas.
  - HII region feedback: gas around young stars that extend out to Stromgren radius is simply set to a temperature of 10^4 K (Baba et al. 2017)

Advanced: Simple Stellar Population Approxm.

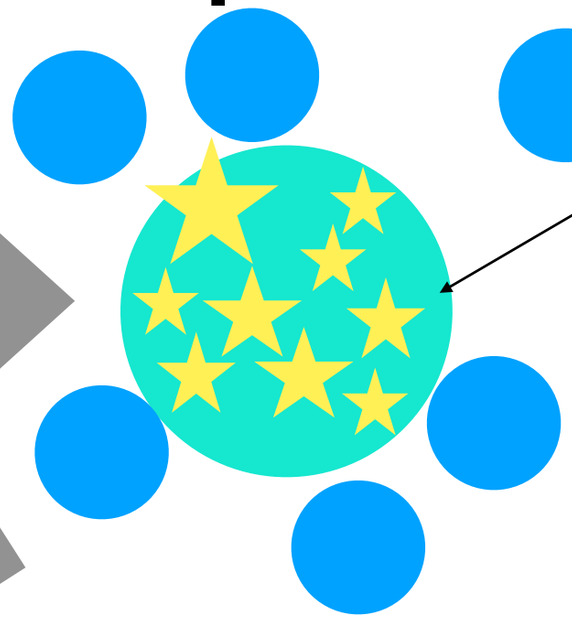
gas (SPH) particle

$\rho_{\rm gas}$
$T_{\rm gas}$
$\nabla \cdot \mathbf{v}$

if $\rho_{\rm gas} > 100$ H/cm^3
$T_{\rm gas} < 100$ K
$\nabla \cdot \mathbf{v} < 0$

Schmidt law:

$$\frac{d\rho_{\rm SF}}{dt} = c_\star \rho_{\rm gas}^{1/2}$$

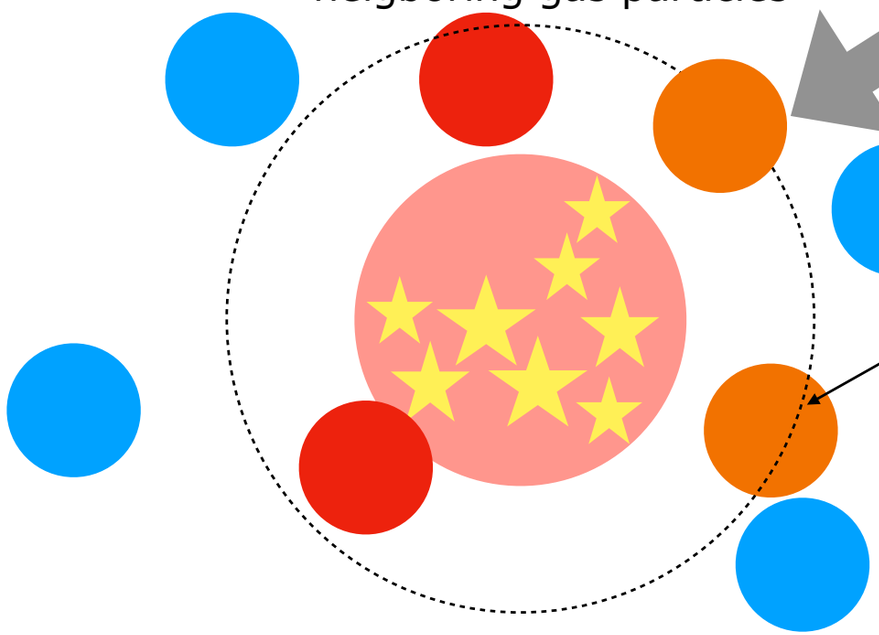


gas particle

star particle
(~ Star Cluster)

Identical Age
Uniform Chemical composition
Salpeter IMF
with a range of 0.1-100 $M_\odot$

neigboring gas particles



OB stars/star particle <~ 1%
==> Hii region, SN II explosion

Heated gas particle
(Feedback from star particle)

Since these models are "ad-hoc", it will be necessary to refine them based on the latest research on ISM processes.